MAT 237

Ensign College

2026

About This Book

This textbook was prepared by Ian Monk for Ensign College and incorporates and adapts Creative Commons–licensed educational resources, including materials licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0). Each chapter includes attribution and licensing information for its original sources.

Consistent with the terms of the applicable Creative Commons licenses, this work is licensed under the Creative Commons Attribution-ShareAlike 4.0 International License (CC BY-SA 4.0).

Contents

Preliminaries

FIXME: Add definitions for “Theorem,” “Lemma,” “Corollary,” and others

Chapter 1

Propositional Logic

1.1 Propositions and Predicates

Compare with Sections 1.1 and 1.2 from Mathematics for Computer Science

Definition 1.1.1. A proposition is a statement (communication) that is either true or false.

For example, both of the following statements are propositions. The first is true, and the second is false.

Proposition. *The sky is blue.*

Proposition. *1 + 1 = 3.*

Being true or false doesn't sound like much of a limitation, but it does exclude statements such as "Wherefore art thou Romeo?" and "Give me an A!" In order to avoid communication issues, we also exclude statements whose truth varies with circumstance such as, "It's five o'clock," or "The stock market will rise tomorrow"; everyone reading these statements would understand their meaning differently!

Sometimes we do want propositions whose truth value can depend on one or more factors. Rather than letting them vary by circumstance, we introduce variables. Each variable must come with a *domain* that defines exactly what values that variable can take. Some examples of possible domains include the real numbers $\mathbb{R}$, the collection of words defined in the 2025 Oxford English Dictionary, and the group of students in a given class. Any set can act as the domain for a variable (we define sets later in Chapter 3). A proposition whose truth value depends on one or more variables is called a *predicate*.

Suppose n varies over the positive integers. Then "n is a perfect square" describes a predicate, since you can't say if it's true or false until you know what the value of the variable n happens to be. Once you know, for example, that n equals 4, the predicate becomes the true proposition "4 is

a perfect square". Remember, nothing says that the proposition has to be true: if the value of n were 5, you would get the false proposition "5 is a perfect square."

1.2 Logical Connectives

Compare with Section 3.1 from Mathematics for Computer Science

In English, we can modify, combine, and relate propositions with words such as "not," "and," "or," "implies," and "if-then." For example, we can combine three propositions into one like this:

If all humans are mortal **and** all Greeks are human, **then** all Greeks are mortal.

We call such a proposition a *compound proposition*, and we call the smaller, indivisible propositions from which it is built *atomic propositions*. So in the examples above, the atomic propositions are "all humans are mortal," "all Greeks are human," and "all Greeks are mortal." Typically we assign a *propositional variable* to each atomic proposition to avoid having cluttered compound propositions, though we will also use propositional variables as placeholders where any proposition could go-compound or atomic. Letting P represent the statement "all humans are mortal," Q represent "all Greeks are human," and R represent "all Greeks are mortal," the above compound proposition becomes

If P **and** Q, **then** R.

Like other propositions, predicates are often named with a letter. Furthermore, a function-like notation is used to denote a predicate supplied with specific variable values. For example, we might use the letter "P" to represent the predicate "n is a perfect square" like so:

$$P(n) := \text{“}n \text{ is a perfect square.”}$$

This notation for predicates is confusingly similar to ordinary function notation. If P is a predicate, then $P(n)$ is either *true* or *false*, depending on the value of n. **Don't confuse these two!**

We will frequently use propositional variables such as P and Q that do not correspond to specific propositions. In some cases, we will specify that the letters have specific meanings, however, most of the time you can think of them as blank spaces where you could insert any proposition—including compound propositions. The understanding is that these empty propositional variables, like propositions, can take on only the values **T** (true) and **F** (false). Propositional variables are also called Boolean variables after their inventor, the nineteenth century mathematician George Boole.

1.2.1 `NOT`, `AND`, and `OR`

Mathematicians use the words `NOT`, `AND` and `OR` for operations that change or combine propositions. The precise mathematical meaning of these special words can be specified by truth tables. For

example, if P is a proposition, then so is "NOT(P)," and the truth value of the proposition "NOT(P)" is determined by the truth value of P according to the following truth table:

P	NOT(P)
T	**F**
F	**T**

The first row of the table indicates that when proposition P is true, the proposition "NOT(P)" is false. The second line indicates that when P is false, "NOT(P)" is true. If P were the proposition. "2 is even," then NOT(P) would be the proposition "2 is not even," or alternatively, "2 is odd." This is probably what you would expect. We call "NOT(P)" the *negation* of P.

In general, a truth table indicates the true/false value of a proposition for each possible set of truth values for the variables. For example, the truth table for the proposition "P AND Q" has four lines, since there are four settings of truth values for the two variables:

P	Q	P AND Q
T	**T**	**T**
T	**F**	**F**
F	**T**	**F**
F	**F**	**F**

According to this table, the proposition "P AND Q" is true only when P and Q are both true. So if P were the proposition, "I am hungry," and Q were the proposition, "I am thirsty", then P AND Q would be the proposition, "I am hungry and thirsty." If I am either not hungry or not thirsty, then the statement "I am hungry and thirsty" would be false. This is probably the way you ordinarily think about the word "and."

There is a subtlety in the truth table for "P OR Q":

P	Q	P OR Q
T	**T**	**T**
T	**F**	**T**
F	**T**	**T**
F	**F**	**F**

The first row of this table says that "P OR Q" is true even if both P and Q are true. This isn't always the intended meaning of "or" in everyday speech, but this is the standard definition in mathematical writing. So if a mathematician says, "You may have cake, or you may have ice cream," he means that you could have both.

If you want to exclude the possibility of having both cake and ice cream, you should combine them with the *exclusive*-or operation, XOR:

P	Q	P `XOR` Q
T	**T**	**F**
T	**F**	**T**
F	**T**	**T**
F	**F**	**F**

Just like using propositional variables to make compound propositions more compact and easy to read, mathematicians use symbols in place of logical connectives to help declutter lengthy compound propositions. This has the added benefit of clearly distinguishing the inclusive `OR` from the exclusive `XOR`. The symbols for `NOT`, `AND`, and `OR`[1] are shown in the table below:

Symbol	Meaning
$\neg P$	`NOT`(P)
$P \wedge Q$	P `AND` Q
$P \vee Q$	P `OR` Q

For the remainder of this text, we will use these symbols instead of their english counterparts.

1.2.2 If and Only If

Mathematicians commonly join propositions in an additional way that doesn't arise in ordinary speech. The proposition "P `if and only if` Q", usually denoted $P \leftrightarrow Q$, asserts that P and Q have the same truth value. Either both are true or both are false.

P	Q	$P \leftrightarrow Q$
T	**T**	**T**
T	**F**	**F**
F	**T**	**F**
F	**F**	**T**

This logical connective is referred to as the *biconditional*. For example, the following biconditional statement is true for every real number x:

$$x^2 - 4 \geq 0 \leftrightarrow |x| \geq 2.$$

For some values of x, *both* inequalities are true. For other values of x, *neither* inequality is true. In every case, however, the proposition as a whole is true.

Another way we like to think about the biconditional statement $P \leftrightarrow Q$ is by saying that P is *necessary* and *sufficient* for Q (or alternatively, that Q is necessary and sufficient for P)[2].

[1] There are also symbols to represent `XOR`, including $\oplus$ and $\veebar$, but the use of `XOR` is uncommon enough that there is no one standard symbol and many authors simply write "XOR"

[2] Since the biconditional just means the two statements have the same truth value, you can always switch the order of the statements without changing the meaning.

Consider the following example. Let P be the proposition, "The baby will stop crying," and Q be the proposition, "the baby is fed." Then the biconditional statement $P \leftrightarrow Q$ would be, "The baby will stop crying if and only if he is fed." In this case, feeding the baby is *necessary* for him to stop crying in the sense that he won't stop crying until he has been fed. Feeding the baby is *sufficient* in the sense that you don't need to do anything else to stop him from crying; feeding is enough.

There is a subtle difference between necessary and sufficient! In the necessary case, the baby may continue to cry after being fed (because he also needs to be changed, etc.), but feeding is one step to helping sooth him. In the sufficient case, it is possible that there are other ways to sooth him, but feeding is enough on its own. Together, these tell us that the baby needs to be fed, and he only needs to be fed; precisely a biconditional!

1.2.3 Implication

Propositions of the form "if P , then Q" are called *implications*. Implication is often rephrased as "P *implies* Q" and written as "$P \to Q$." Its meaning is the least intuitive of the logical connectives we have covered thus far. Here is its truth table, with the lines labeled so we can refer to them later.

P	Q	$P \to Q$	
T	**T**	**T**	(tt)
T	**F**	**F**	(tf)
F	**T**	**T**	(ft)
F	**F**	**T**	(ff)

The truth table for implications can be summarized in words as follows:

> An implication is true exactly when the if-part (the *hypothesis*) is false or the then-part (the *conclusion*) is true.

This sentence is worth remembering; a large fraction of all mathematical statements are of the if-then form!

Let's experiment with this definition. For example, is the following proposition true or false?

> If Goldbach's Conjecture is true, then $x^2 \geq 0$ for every real number x.

No one knows whether Goldbach's Conjecture, a famous unsolved problem in mathematics, is true or false. But that doesn't prevent us from answering the question! This proposition has the form "if P, then Q" where the hypothesis, P, is "Goldbach's Conjecture is true" and the conclusion, Q, is "$x^2 \geq 0$ for every real number x." Since the conclusion is definitely true, we're on either line (tt) or line (ft) of the truth table. Either way, the proposition as a whole is true! When the conclusion of an implication is true, regardless of the truth value of the hypothesis, we say the implication is *trivially true*.

Now let's figure out the truth of another example:

If pigs fly, then your account won't get hacked.

Forget about pigs, we just need to figure out whether this proposition is true or false. Pigs do not fly, so we're on either line (ft) or line (ff) of the truth table. In both cases, the proposition is true! When the hypothesis of an implication is false, regardless of the truth value of the conclusion, we say the implication is *vacuously true*.

Note that an implication can be both *trivially* and *vacuously* true at the same time as in the following example:

If Superman is real, then the sky is blue.

Since Superman is not real, this statement is vacuously true, but since the sky is blue, this statement is also trivially true.

False Hypotheses

This mathematical convention—that an implication as a whole is considered true when its hypothesis is false—contrasts with common cases where implications are supposed to have some cause-and-effect connection between their hypotheses and conclusions. For example, we could agree—or at least hope—that the following statement is true:

If you followed the security protocol, then your account won't get hacked.

We regard this implication as unproblematic because of the clear causal connection between security protocols and account hackability. On the other hand, the statement:

If pigs could fly, then your account won't get hacked,

would commonly be rejected as false—or at least silly—because porcine aeronautics have nothing to do with your account security. But mathematically, this implication counts as true.

It's important to accept the fact that mathematical implications ignore causal connections. This makes them a lot simpler than causal implications, but useful nevertheless. To illustrate this, suppose we have a system specification which consists of a series of, say, a dozen rules,

If	the	system sensors are in condition 1,
	then	the system takes action 1.
If	the	system sensors are in condition 2,
	then	the system takes action 2.
		⋮
If	the	system sensors are in condition 12,
	then	the system takes action 12.

Letting C_i be the proposition that the system sensors are in condition i, and A_i be the proposition that system takes action i, the specification can be restated more concisely by the logical formulas

$$\begin{aligned} C_1 &\to A_1, \\ C_2 &\to A_2, \\ &\vdots \\ C_{12} &\to A_{12}. \end{aligned}$$

Now the proposition that the system obeys the specification can be nicely expressed as a single logical formula by combining the formulas together with `AND`s:

$$[C_1 \to A_1] \wedge [C_2 \to A_2] \wedge \cdots \wedge [C_{12} \to A_{12}] \tag{1.1}$$

For example, suppose only conditions C_2 and C_5 are true, and the system indeed takes the specified actions A_2 and A_5. So in this case, the system is behaving according to specification, and we accordingly want formula (1.1) to come out true. The implications $C_2 \to A_2$ and $C_5 \to A_5$ are both true because both their hypotheses and their conclusions are true. But in order for (1.1) to be true, we need all the other implications, all of whose hypotheses are false, to be true. This is exactly what the rule for mathematical implications accomplishes.

Converse, Inverse, and Contrapositive

When working with an implication, there are three ways we can rearrange it in order to get more information. We call these the *inverse*, *converse*, and *contrapositive* of the original implication.

Consider the following implication: "If it rains, the sidewalk will be wet." If we look outside and see that the sidewalk is wet, can we conclude that it rained? It is possible that it rained, but it is also possible that somebody was using their garden hose to get the sidewalk wet, some children were having a water balloon fight, or that it had snowed and the snow is melting.

When you switch the hypothesis and the conclusion of an implication, the resulting inplication is called the *converse*. So the converse of the implication, "If it rains, the sidewalk will be wet," would be the implication, "If the sidewalk is wet, then it rained." Just because an implication is true does not mean that the converse is true, however it is possible for both an implication and its converse to be true simultaneously. That being said, knowing the truth value of an implication gives you no information about the truth of its converse. This is an easy mistake to make since in English we often use "If... then..." statements when we actually mean "if and only if."

Knowing that rain leads to wetness, can we conclude that if the sidewalk is *not* wet that it did *not* rain? Yes! Since we know that rain must lead to a wet sidewalk, no wetness means no rain. Changing the order of the hypothesis and the conclusion *and negating both* is called the *contrapositive*, and it always has the same truth value as the original implication. So the contrapositive of the implication, "If it rains, the sidewalk will be wet," is the implication, "If the

sidewalk is not wet, then it did not rain." This will become more important later when we discuss proof techniques involving implications.

There is one other way we often modify implications to get new information. If we negate both the hypothesis and the conclusion but do not swap the order, we get what is called the *inverse*. So the inverse of the implication, "If it rains, the sidewalk will be wet," would be the implication, "If it does not rain, the sidewalk will not be wet." Like the converse, the inverse is not necessarily true when the original implication is true because there are other possible events that may result in the sidewalk being wet even without rain. In fact, just like how the original implication and the contrapositive always have the same truth value, the converse and the inverse always have the same truth value[3].

Table 1.1 below summarizes the four implication variants.

Implication	Name	Has the same meaning as
$P \to Q$	Original	Contrapositive
$Q \to P$	Converse	Inverse
$\neg P \to \neg Q$	Inverse	Converse
$\neg Q \to \neg P$	Contrapositive	Original

Table 1.1: The four implication variants

Ways to State Implications

There are many ways to state an implication in English. We have seen a few already: "If P, then Q" and "If P, Q." Since implication is very precise and nuanced, mathematicians use specific terminology to discuss implications to make sure the meaning comes across clearly. Table 1.2 below lists the most common of these.

If P, then Q	P implies Q
Q if P	P only if Q
If P, Q	Only if Q, P
Q when P	Q unless $\neg P$
Q whenever P	Q follows from P
P is sufficient for Q	Q is necessary for P
A sufficient condition for Q is P	A necessary condition for P is Q

Table 1.2: Different phrasings of $P \to Q$ in English

Notice how some of this terminology is similar to the phrases used to describe biconditionals. That is not a conicidence! Consider the compound proposition $(P \to Q) \wedge (Q \to P)$. Some ways we could read this include "P only if Q and P if Q" or as "P is sufficient for Q and P is necessary

[3] One way to see this is to notice that the inverse is the contrapositive of the converse. Another way would simply be to construct truth tables for each of the four implications and see which ones match.

for Q." We often abbreviate these as "P if and only if Q" and "P is necessary and sufficient for Q." This is exactly the terminology used to discuss biconditionals! As it turns out, the statement $(P \to Q) \wedge (Q \to P)$ has the exact same truth table as $P \leftrightarrow Q$ and therefore has the same meaning.

1.3 Quantifiers

Compare with Section 3.6 from Mathematics for Computer Science

1.3.1 For All and Exists

The predicate

$$\text{“}x^2 \geq 0\text{”}$$

is always true when x is a real number. We use the "for all" symbol $\forall$ to indicate this, and we call the set of real numbers $\mathbb{R}$ the *domain* for the variable x. To indicate that the domain of x is the real numbers, we use the element symbol $\in$ and say $x \in \mathbb{R}$. Putting all this together, we get that the proposition

$$\forall\, x \in \mathbb{R}, x^2 \geq 0$$

is a true statement. This would be read as, "for all x in the real numbers, $x^2 \geq 0$." On the other hand, the predicate

$$\text{“}5x^2 - 7 = 0\text{”}$$

is only sometimes true; specifically, when $x = \pm\sqrt{7/5}$. There is a "there exists" notation $\exists$ to indicate that a predicate is true for at least one, but not necessarily all objects. So

$$\exists\, x \in \mathbb{R} : 5x^2 - 7 = 0$$

(read as "there exists x in the real numbers such that $5x^2 - 7 = 0$") is true, while

$$\forall\, x \in \mathbb{R} : 5x^2 - 7 = 0$$

is not true.

There are several ways to express the notions of "always true" and "sometimes true" in English. The table below gives some general formats on the left[4] and specific examples using those formats on the right. You can expect to see such phrases hundreds of times in mathematical writing!

[4] In the left column, we use D to indicate a generic set. For now picture your favorite set (the real numbers, the integers, etc.). We will discuss sets in more detail in Chapter 3.

Always True	
For all $x \in D$, $P(x)$ is true.	For all $x \in \mathbb{R}$, $x^2 \geq 0$.
$P(x)$ is true for every x in the set D.	$x^2 \geq 0$ for every $x \in \mathbb{R}$.
Sometimes True	
There is an $x \in D$ such that $P(x)$ is true.	There is an $x \in \mathbb{R}$ such that $5x^2 - 7 = 0$.
$P(x)$ is true for some x in the set D.	$5x^2 - 7 = 0$ for some $x \in \mathbb{R}$.
$P(x)$ is true for at least one $x \in D$.	$5x^2 - 7 = 0$ for at least one $x \in \mathbb{R}$.

All these sentences "quantify' how often the predicate is true. Specifically, an assertion that a predicate is always true is called a *universal quantification*, and an assertion that a predicate is sometimes true is an *existential quantification*. Sometimes the English sentences are unclear with respect to quantification:

$$\text{If you can solve any problem we come up with,}$$
$$\text{then you get an A for the course.} \tag{1.2}$$

The phrase "you can solve any problem we can come up with" could reasonably be interpreted as either a universal or existential quantification:

$$\text{you can solve } \textit{every} \text{ problem we come up with,} \tag{1.3}$$

or maybe

$$\text{you can solve } \textit{at least one} \text{ problem we come up with.} \tag{1.4}$$

To be precise, let $\texttt{Probs}$ be the set of problems we come up with, $\texttt{Solves}(x)$ be the predicate "You can solve problem x," and G be the proposition, "You get an A for the course." Then the two different interpretations of (1.2) can be written as follows:

$$(\forall\, x \in \texttt{Probs}, \texttt{Solves}(x)) \to G, \qquad \text{for (1.3)},$$
$$(\exists\, x \in \texttt{Probs}, \texttt{Solves}(x)) \to G. \qquad \text{for (1.4)}.$$

1.3.2 Mixing Quantifiers

Many mathematical statements involve several quantifiers. For example,

Goldbach's Conjecture: Every even integer greater than 2 is the sum of two primes.

Let's write this out in more detail to be precise about the quantification:

For every even integer n greater than 2, there exist primes p and q such that $n = p + q$.

Let `Evens` be the set of even integers greater than 2, and let `Primes` be the set of primes. Then we can write Goldbach's Conjecture in logic notation as follows:

$$\underbrace{\forall\, n \in \texttt{Evens}}_{\substack{\text{for every even}\\ \text{integer } n > 2}} \underbrace{\exists\, p \in \texttt{Primes}\ \exists\, q \in \texttt{Primes}}_{\substack{\text{there exist primes}\\ p \text{ and } q \text{ such that}}}, n = p + q.$$

1.3.3 Order of Quantifiers

Swapping the order of different kinds of quantifiers (existential or universal) usually changes the meaning of a proposition. For example:

"Every American has a dream."

This sentence is ambiguous because the order of quantifiers is unclear. Let A be the set of Americans, let D be the set of dreams, and define the predicate $H(a, d)$ to be "American a has dream d." Now the sentence could mean there is a single dream that every American shares—such as the dream of owning their own home:

$$\exists\, d \in D\ \forall\, a \in A, H(a, d)$$

Or it could mean that every American has a personal dream:

$$\forall\, a \in A\ \exists\, d \in D, H(a, d)$$

For example, some Americans may dream of a peaceful retirement, while others dream of continuing practicing their profession as long as they live, and still others may dream of being so rich they needn't think about work at all.

Swapping quantifiers in Goldbach's Conjecture creates a patently false statement that every even number ≥ 2 is the sum of *the same* two primes:

$$\underbrace{\exists\, p \in \texttt{Primes}\ \exists\, q \in \texttt{primes}}_{\substack{\text{there exist primes}\\ p \text{ and } q \text{ such that}}}, \underbrace{\forall\, n \in \texttt{Evens}}_{\substack{\text{for every even}\\ \text{integer } n > 2}} n = p + q.$$

1.3.4 Negating Quantifiers

There is a simple relationship between the two kinds of quantifiers. The following two sentences mean the same thing:

Not everyone likes ice cream.
There is someone who does not like ice cream.

The equivalence of these sentences is an instance of a general equivalence that holds between predicate formulas:

$$\neg(\forall\, x, P(x)) \quad \text{is equivalent to} \quad \exists\, x, \neg P(x) \tag{1.5}$$

Similarly, these sentences mean the same thing:

There is no one who likes being mocked.
Everyone dislikes being mocked.

The corresponding predicate formula equivalence is

$$\neg(\exists\, x, P(x)) \quad \text{is equivalent to} \quad \forall\, x, \neg P(x). \tag{1.6}$$

Note that the equivalence (1.6) follows directly by negating both sides the equivalence (1.5).

The general principle is that moving a $\neg$ to the other side of an $\exists$ changes it into a $\forall$, and vice versa.

These equivalences are called *De Morgan's Laws for Quantifiers* because they can be understood as applying the De Morgan's Law rules of inference (see Section 1.4) to an infinite sequence of $\wedge$'s and $\vee$'s. For example, we can explain (1.5) by supposing the domain of x is $\{d_0, d_1, \ldots, d_n, \ldots\}$. Then $\exists\, x, \neg P(x)$ means the same thing as the infinite `OR`:

$$\neg P(d_0) \vee \neg P(d_1) \vee \cdots \vee \neg P(d_n) \vee \ldots. \tag{1.7}$$

Applying De Morgan's rule to this infinite `OR` yields the equivalent formula

$$\neg[P(d_0) \wedge P(d_1) \wedge \cdots \wedge P(d_n) \wedge \ldots]. \tag{1.8}$$

But (1.8) means the same thing as

$$\neg[\forall\, x, P(x)].$$

This explains why $\exists\, x, \neg P(x)$ means the same thing as $\neg[\forall\, x, P(x))]$, which confirms (1.5).

1.4 Rules of Inference

Compare with Section 1.4.1 from Mathematics for Computer Science

Logical deductions, or *rules of inference*, are used to prove new propositions using previously proved ones.

A fundamental inference rule is *modus ponens*. This rule says that if we know P is true and we know that $P \to Q$ is true, then we also know that Q is true.

Rules of inference are sometimes written in a funny notation. For example, *modus ponens* is written:

$$\frac{\begin{array}{l}P\\P \to Q\end{array}}{Q}$$

When the statements above the line, called the *antecedents*, are true, then we can consider the statement below the line, called the *consequent*, to also be true.

A key requirement of a rule of inference is that it must be *sound*: an assignment of truth values to the letters P, Q, ..., that makes all the antecedents true must also make the consequent true.

There are many examples of rules of inference, but some common ones are shown in Table 1.3 below.

Rule of Inference	Name	Rule of Inference	Name
$\dfrac{\begin{array}{l}P\\P \to Q\end{array}}{Q}$	Modus Ponens	$\dfrac{P}{P \vee Q}$	Addition
$\dfrac{\begin{array}{l}\neg Q\\P \to Q\end{array}}{\neg P}$	Modus Tollens	$\dfrac{P \wedge Q}{P}$	Simplification
$\dfrac{\begin{array}{l}P \to Q\\Q \to R\end{array}}{P \to R}$	Hypothetical Syllogism	$\dfrac{\begin{array}{l}P\\Q\end{array}}{P \wedge Q}$	Adjunction
$\dfrac{\begin{array}{l}P \vee Q\\\neg P\end{array}}{Q}$	Disjunctive Syllogism	$\dfrac{\begin{array}{l}P \vee Q\\\neg P \vee R\end{array}}{Q \vee R}$	Resolution

Table 1.3: Some common rules of inference

Notice that the rules of inference only go one way. For example, even if we know Q is true, we cannot use modus ponens to conclude that P and $P \to Q$ are both true[5]. A rule of inference that goes both directions is called a *logical equivalence*. If two propositions are logically equivalent, it means they always share the *same* truth value, either both true or both false. We use the notation $P \equiv Q$ to indicate that P and Q are logically equivalent. This is identical to the biconditional $\leftrightarrow$ discussed in Section 1.2.2, however mathematicians typically use the notation $\leftrightarrow$ as a logical connective like $\wedge$, $\vee$, and $\to$ to make a *single* compound proposition but use $\equiv$ to emphasize that two *distinct* propositions always share a truth value.

[5]We can't conclude that P is true from Q without additional information, but we can actually conclude that $P \to Q$ is true from just Q using a different rule of inference. In this case, we would say that $P \to Q$ is *trivially* true. Contrapositively, if we know P is false, we conclude that $P \to Q$ is *vacuously* true.

Some typical logical equivalences are shown in Table 1.4 below. Take particular note of De Morgan's Laws as they will show up in another form when we discuss sets later.

Equivalence	Name
$\neg(\neg P) \equiv P$	Double Negation Law
$P \wedge \mathbf{T} \equiv P$ $P \vee \mathbf{F} \equiv P$	Identity Laws
$P \wedge \mathbf{F} \equiv \mathbf{F}$ $P \vee \mathbf{T} \equiv \mathbf{T}$	Domination Laws
$P \vee \neg P \equiv \mathbf{T}$ $P \wedge \neg P \equiv \mathbf{F}$	Negation Laws
$P \vee P \equiv P$ $P \wedge P \equiv P$	Idempotent Laws
$P \vee Q \equiv Q \vee P$ $P \wedge Q \equiv Q \wedge P$	Commutative Laws
$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$	Associative Laws
$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (Q \vee R)$ $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (Q \wedge R)$	Distributive Laws
$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$ $\neg(P \wedge Q) \equiv \neg P \vee \neg Q$	De Morgan's Laws
$P \vee (P \wedge Q) \equiv P$ $P \wedge (P \vee Q) \equiv P$	Absorption Laws
$P \to Q \equiv \neg P \vee Q$	Implication Law
$P \to Q \equiv \neg Q \to \neg P$	Contrapositive Law
$P \leftrightarrow Q \equiv (P \to Q) \wedge (Q \to P)$	Biconditional Law

Table 1.4: Some common logical equivalences

1.4.1 Inference with Quantifiers

In addition to the above rules of inference, there are also special rules of inference that deal with quantification.

Rule of Inference	Name
$\dfrac{\forall\, x, P(x)}{P(s)}$	Universal Instantiation
$\dfrac{P(a)\quad a \text{ is arbitrary}}{\forall\, x, P(x)}$	Universal Generalization
$\dfrac{\exists\, x, P(x)}{P(u) \text{ for some } u}$	Existential Instantiation
$\dfrac{P(s)}{\exists\, x, P(x)}$	Existential Generalization

Table 1.5: The four basic quantified rules of inference. Within the domain of x, the elements a, s, and u are arbitrary, specific, and unknown, respectively.

We will go through each of these explaining their nuances.

Universal Instantiation

Universal Instantiation says that if a predicate is true for *all* elements of the domain of a variable, then you can substitute *any* element of the domain and it will still be true.

For example, let P be the predicate $P(x) := x > 0$ where the domain of x is the positive integers. Since all positive numbers are strictly greater than zero (zero is considered neither positive nor negative), $P(x)$ must be true for all positive integers. Hence, $\forall x, P(x)$ is a true statement. Since 5 is a positive integer, we can conclude by Universal Instantiation that $P(5)$ is also true.

Universal Generalization

Universal Generalization is a bit more complicated than Universal Instantiation. Suppose that you have a predicate $P(x)$ where x has domain D. Our goal here is to prove that $P(x)$ is true no matter which element of D we substitute for x. In order to do this, we are going to let a be an arbitrary element of D.

What does it mean for an element to be *arbitrary*?

We use the word "arbitrary" to mean that we are not allowed to assume anything about a except the things that would be true for *any* element of D. This is the opposite of taking a *specific* element of D.

We are going to do both an example and a non-example to help illustrate exactly what Universal Generalization *is* and what it *is not*.

Example: We are going to prove that adding 2 to any even integer results in another even integer.

Let a be an arbitrary even integer. Since a is even, it is divisible by 2. This means that $a = 2 \times b$ for some unknown integer b. Adding 2 gives us $a+2 = 2b+2 = 2(b+1)$. Since $b+1$ is an integer, $2(b+1)$ is an even integer. Since $a + 2 = 2(b + 1)$, we get that $a + 2$ is even.

Since a was an arbitrary even integer, we get by Universal Generalization that the result is true for all even integers.

Non-example: We are going to prove that all integers are even. We pick integers: 2, -4056, and 876543210. In each case, we have an even number. Since our choice was arbitrary, we get by Universal Generalization that all integers are even.

Notice how **arbitrary does not mean random**. Checking a few randomly selected cases is not enough to make a generalization about all elements of a domain. The above non-example shows how doing this would allow us to "prove" claims that are completely false. To work with an arbitrary element of the domain, you must assign it a letter variable and use only properties that apply to *every* element of the domain.

Existential Instantiation

Existential Instantiation says that if we know a predicate is true for *some* (at least one) element of a domain, we can pick an element out of the domain for which the predicate is true. We don't know which element(s) actually work; we only know that there is at least one in the domain somewhere. Similar to picking an arbitrary element as in Universal Generalization, we let a variable represent such an element. When working with this variable, we are allowed to use:

1. Properties that apply to every element of the domain
2. The fact that the proposition is true when we substitute in the variable

As an example, let P be the predicate $P(x) :=$ "x gave Sam a gift" and let Q be the proposition $Q :=$ "Sam is happy". Sam likes receiving gifts from anyone, so $\forall\, x(P(x) \to Q)$ is a true statement.

Suppose we see a gift in Sam's hands. This means $\exists\, x, P(x)$ is true. By Existential Instantiation, we can call the mystery gifter g and conclude that $P(g)$ is true. By Universal Instantiation, $P(g) \to Q$, so Modus Ponens in turn gives us that Q. Hence, we can conclude that Sam is happy. We don't know *who* made Sam happy, but we are still able to conclude that he *is* happy.

Existential Generalization

Exestential Generalization is quite simple. It says that if we can prove a predicate is true for a *specific value*, then there exists a value for which it is true (namely, the one we proved works, though there could be others). For example, letting P be the predicate $P(x) :=$ "x is even", where the domain of x is the integers, we can see immediately that $P(2)$ is true. Hence, by Existential Generalization, $\exists\, x, P(x)$ is true.

Chapter 2

Proof Techniques

In principle, a proof can be *any* sequence of logical deductions from axioms and previously proved statements that concludes with the proposition in question. This freedom in constructing a proof can seem overwhelming at first. How do you even *start* a proof?

Here's the good news: many proofs follow one of a handful of standard templates. Each proof has it own details, of course, but these templates at least provide you with an outline to fill in. We'll go through several of these standard patterns, pointing out the basic idea and common pitfalls and giving some examples. Many of these templates fit together; one may give you a top-level outline while others help you at the next level of detail. And we'll show you other, more sophisticated proof techniques later on.

The recipes below are very specific at times, telling you exactly which words to write down on your piece of paper. You're certainly free to say things your own way instead; we're just giving you something you *could* say so that you're never at a complete loss.

2.1 Proofs in Propositional Logic

2.1.1 Proof by Truth Table

Before, we used truth tables to define logical connectives, however their usefulness goes beyond definitions. If you want to prove a proposition is true, it is sufficient to show it is true in every situation. For example, consider the statement, "$P \wedge \neg P$ is false." This is equivalent to $\neg(P \wedge \neg(P))$, a proof of which is shown below:

P	$\neg P$	$P \wedge \neg P$	$\neg(P \wedge \neg P)$
T	**F**	**F**	**T**
F	**T**	**F**	**T**

So no matter what truth value is chosen for P, the statement $\neg(P \wedge \neg P)$ will always be true. A

proposition that is always true regardless of the truth values of its propositional variables is called a *tautology*. Similarly, a proposition that is always false is called a *contradiction*.

Any rule of inference that is not a logical equivalence can be written in propositional logic as "if all the antecedents are true, the consequent is true." In other words, we can take the conjunction of the antecedents as the hypothesis of an implication with the consequent as the conclusion.

For example, consider Modus Ponens. Written in logic terms, it would be $[P \wedge (P \to Q)] \to Q$. If we can show that this statement is a tautology, then we will have proven Modus Ponens. An example of such a truth table is shown below:

P	Q	$P \to Q$	$P \wedge (P \to Q)$	$[P \wedge (P \to Q)] \to Q$
T	**T**	**T**	**T**	**T**
T	**F**	**F**	**F**	**T**
F	**T**	**T**	**F**	**T**
F	**F**	**T**	**F**	**T**

Note that each column of the above truth table other than the ones containing the propositional variables only adds exactly *one* logical operator to some combination of columns that already exist. Column 3 is an implication from column 1 to column 2, column 4 is the conjunction of columns 1 and 3, and column 5 is an implication from column 4 to column 2. This is to make the table easy to read for someone seeking to understand *why* Modus Ponens is true. As an example of why this is important, consider the following Modus Ponens truth table:

P	Q	$[P \wedge (P \to Q)] \to Q$
T	**T**	**T**
T	**F**	**T**
F	**T**	**T**
F	**F**	**T**

The table shows that the author believes Modus Ponens to be true, but does not offer any additional insights as to *why* it is true or *how* the author came to that conclusion. When doing a proof by truth table, it is considered good form to break each logical operator into its own column.

Recall that logical equivalence "$\equiv$" tells us the same information as the biconditional "$\leftrightarrow$." This means that logical equivalences can be rephrased as tautologies that use $\leftrightarrow$ to connect two propositions. More often than not, however, we omit the column with the biconditional and instead check by observation that the columns for each proposition have matching truth values. Here is an example proving the Implication Law:

P	Q	$\neg P$	$P \to Q$	$\neg P \vee Q$
T	**T**	**F**	**T**	**T**
T	**F**	**F**	**F**	**F**
F	**T**	**T**	**T**	**T**
F	**F**	**T**	**T**	**T**

Since both the $P \to Q$ column and the $\neg P \vee Q$ column have matching truth values, we have proven that $P \to Q \equiv \neg P \vee Q$.

Truth tables do have one drawback: they quickly get *very* large. To illustrate this, consider the truth table below proving Hypothetical Syllogism:

P	Q	R	$P \to Q$	$Q \to R$	$P \to R$	$(P \to Q) \wedge (Q \to R)$	$[(P \to Q) \wedge (Q \to R)] \to (P \to R)$
T	**T**	**T**	**T**	**T**	**T**	**T**	**T**
T	**T**	**F**	**T**	**F**	**F**	**F**	**T**
T	**F**	**T**	**F**	**T**	**T**	**F**	**T**
T	**F**	**F**	**F**	**T**	**F**	**F**	**T**
F	**T**	**T**	**T**	**T**	**T**	**T**	**T**
F	**T**	**F**	**T**	**F**	**T**	**F**	**T**
F	**F**	**T**	**T**	**T**	**T**	**T**	**T**
F	**F**	**F**	**T**	**T**	**T**	**T**	**T**

That's a big table! In order to get every combination of truth values of the 3 propositional variables P, Q, and R, we need $2^3 = 8$ rows. This pattern continues, with the number of rows doubling with each new propositional variable added. For this reason, proof by truth table is usually limited to simple statements with few propositional variables. To prove more complicated propositions, other techniques such as formal proofs are usually favored.

2.1.2 Formal Proofs

A *formal proof* is a special kind of proof used when working with propositional logic. All good proofs should be "formal" in the sense that they are rigorous and don't leave out important details, but in this book we will use the phrase "formal proof" specifically to refer to the particular format described below.

Most proofs using propositional logic will ask you to prove a result using a specific list of premises. When writing a formal proof, we assume the premises are true and use rules of inference until we reach the desired conclusion.

We use a very specific structure when writing a formal proof:

Proof.

1.	Conclusion 1	(Justification 1)
2.	Conclusion 2	(Justification 2)
3.	Conclusion 3	(Justification 3)
⋮	⋮	⋮

□

For example, consider the following formal proof of Hypothetical Syllogism. We take the antecedents $P \to Q$ and $Q \to R$ as premises, and use rules of inference on them until we are able to conclude $P \to R$.

Proof.

1.	$P \to Q$	(Premise)
2.	$\neg P \vee Q$	(Implication Law on 1)
3.	$Q \to R$	(Premise)
4.	$\neg Q \vee R$	(Implication Law on 3)
5.	$Q \vee \neg P$	(Commutativity on 2)
6.	$\neg P \vee R$	(Resolution on 5 & 4)
7.	$P \to R$	(Implication Law on 6)

□

That's a lot nicer than the truth table proof from Section 2.1.1! Notice how each justification references exactly which lines are being used as the antecedents for the rule of inference mentioned. Also notice how row 5 applies commutativity so that the proposition is in *exactly* the right format to apply resolution in the next step. While it is common to do multiple instances of commutativity or associativity in one step, a good rule of thumb is that it is better to be too detailed than not detailed enough!

Proving an Equivalence

When using a formal proof to prove a logical equivalence, we pick one side of the equivalence (your choice) to use as the premise and use known logical equivalences until we conclude the other side. One thing to note is that when proving a logical equivalence by a formal proof, *every justification must also be a logical equivalence*[1]. This means that any equivalence from Table 1.4 would make a valid justification, but none of the rules of inference from Table 1.3 or Table 1.5 would be valid.

Consider the following example:

Theorem 2.1.1. *The negation of $P \to Q$ is $P \wedge \neg Q$.*

This could be rephrased in the terms of propositional logic as "$\neg(P \to Q) \equiv P \wedge \neg Q$."

[1] Remembering that $\equiv$ means the same thing as $\leftrightarrow$, you could do a biconditional proof as in Section 2.4 to prove an equivalence by two implications, each of which would allow you to use regular rules of inference. However, this would no longer be considered a *formal proof* as defined here.

Proof.

1.	$\neg(P \to Q)$	(Premise)
2.	$\neg(\neg P \vee Q)$	(Implication Law on 1)
3.	$\neg(\neg P) \wedge \neg Q$	(De Morgan's Law on 2)
4.	$P \wedge \neg Q$	(Double Negation Law on 3)

□

Since each step was by logical equivalence, we can reverse the steps and get an equally valid proof of Theorem 2.1.1:

Proof.

1.	$P \wedge \neg Q$	(Premise)
2.	$\neg(\neg P) \wedge \neg Q$	(Double Negation Law on 1)
3.	$\neg(\neg P \vee Q)$	(De Morgan's Law on 2)
4.	$\neg(P \to Q)$	(Implication Law on 3)

□

The second approach should feel less intuitive since you have to substitute $\neg(\neg P)$ for P, which is not an obvious first step. With many equivalence proofs, you will have to use logical equivalences to "complexify" rather than simplify. However, since you can take either side of the equivalence as your starting premise, you can always start on both ends and simplify until you meet in the middle.

For example, consider the Contrapositive Law, $P \to Q \equiv \neg Q \to \neg P$. Starting with the left, we can apply the Implication Law to $P \to Q$ to get $\neg P \vee Q$, but where can we go from there? Since the next step is not obvious, let's try starting with the right and see where that takes us. Applying the Implication Law to $\neg Q \to \neg P$ gives us $\neg(\neg Q) \vee \neg P$. The Double Negation Law then yields $Q \vee \neg P$. Since $\vee$ is commutative, is the same result we got when we started with $P \to Q$! Putting all this together, we get the following proof of the Contrapositive Law:

Proof.

1.	$P \to Q$	(Premise)
2.	$\neg P \vee Q$	(Implication Law on 1)
3.	$Q \vee \neg P$	(Commutativity on 2)
4.	$\neg(\neg Q) \vee \neg P$	(Double Negation on 3)
5.	$\neg Q \to \neg P$	(Implication Law on 4)

□

2.2 Proof by Contradiction

Compare with Section 1.8 from Mathematics for Computer Science

In a *proof by contradiction*, or *indirect proof*, you show that if a proposition were false, then some false fact would be true. Since a false fact by definition can't be true, the proposition must be true.

Proof by contradiction is *always* a viable approach. However, as the name suggests, indirect proofs can be a little convoluted, so direct proofs are generally preferable when they are available.

Method: In order to prove a proposition P by contradiction:

1. Write, "We use proof by contradiction."
2. Write, "Suppose $\neg P$ is true."
3. Deduce something known to be false (a logical contradiction).
4. Write, "This is a contradiction, therefore, P must be true."

If P is a compound proposition, you will need to use logical equivalences to compute its negation. Such a computation will closely mirror the formal proofs discussed in Section 2.1.2.

Example

We'll prove by contradiction that $\sqrt{2}$ is irrational. Remember that a number is rational if it is equal to a ratio of integers—for example, $3.5 = \frac{7}{2}$ and $0.1111\cdots = \frac{1}{9}$ are rational numbers.

Theorem 2.2.1. *$\sqrt{2}$ is irrational.*

Proof. We use proof by contradiction. Suppose the claim is false, and $\sqrt{2}$ is rational. Then we can write $\sqrt{2}$ as a fraction $\frac{n}{d}$ in lowest terms[2]. Squaring both sides gives $2 = \frac{n^2}{d^2}$ and so multiplying both sides by d^2 gives us $2d^2 = n^2$. This implies that n^2 is a multiple of 2, so n is a multiple of 2. Writing $n = 2k$ for some integer k, we get that $2d^2 = (2k)^2 = 4k^2$. Dividing both sides by 2 gives us $d^2 = 2k^2$. By a similar argument as before, this implies that d is a multiple of 2. So, n and d have 2 as a common factor, which contradicts the fact that $\frac{n}{d}$ is in lowest terms. Thus, $\sqrt{2}$ must be irrational. □

2.3 Proving an Implication

Compare with Section 1.5 from Mathematics for Computer Science

Propositions of the form "If P, then Q" are called implications. This implication is often rephrased as "$P \to Q$."

Here are some examples:

[2]Lowest terms means that n and d share no common factors. For example, $\frac{2}{4}$ is not in lowest terms, but $\frac{1}{2}$ is.

- (Quadratic Formula) If $ax^2 + bx + c = 0$, then

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

- (Goldbach's Conjecture, rephrased) If n is an even integer greater than 2, then n is the sum of two primes.
- If $0 \leq x \leq 2$, then $-x^3 + 4x + 1 > 0$.

There are a couple of standard methods for proving an implication.

2.3.1 Proving an Implication Directly

In order to prove that $P \to Q$:

1. Write, "Assume P."
2. Show that Q logically follows.

Example

Theorem 2.3.1. *If $0 \leq x \leq 2$, then $-x^3 + 4x + 1 > 0$.*

Before we write a proof of this theorem, we have to do some scratchwork to figure out why it is true.

The inequality certainly holds for $x = 0$; then the left side is equal to 1 and $1 > 0$. As x grows, the $4x$ term (which is positive) initially seems to have greater magnitude than $-x^3$ (which is negative). For example, when $x = 1$, we have $4x = 4$, but $-x^3 = -1$ only. In fact, it looks like $-x^3$ doesn't begin to dominate until $x > 2$. So it seems the $-x^3 + 4x$ part should be nonnegative for all x between 0 and 2, which would imply that $-x^3 + 4x + 1$ is positive.

So far, so good. But we still have to replace all those "seems like" phrases with solid, logical arguments. We can get a better handle on the critical $-x^3 + 4x$ part by factoring it, which is not too hard:

$$-x^3 + 4x = x(2 - x)(2 + x)$$

Aha! For x between 0 and 2, all of the terms on the right side are nonnegative. And a product of nonnegative terms is also nonnegative. Let's organize this blizzard of observations into a clean proof.

Proof. Assume $0 \leq x \leq 2$. Then x, $2 - x$ and $2 + x$ are all nonnegative. Therefore, the product of these terms is also nonnegative. Adding 1 to this product gives a positive number, so:

$$x(2 - x)(x + x) + 1 > 0$$

Multiplying out on the left side proves that

$$-x^3 + 4x + 1 > 0$$

as claimed. □

There are a couple points here that apply to all proofs:

- You'll often need to do some scratchwork while you're trying to figure out the logical steps of a proof. Your scratchwork can be as disorganized as you like: full of dead-ends, strange diagrams, obscene words, whatever. But keep your scratchwork separate from your final proof, which should be clear and concise.
- Proofs typically begin with the word "Proof" and end with some sort of delimiter like □ or "QED." The only purpose for these conventions is to clarify where proofs begin and end.

2.3.2 Proving the Contrapositive

An implication ("$P \to Q$") is logically equivalent to its contrapositive

$$\neg Q \to \neg P.$$

Proving one is as good as proving the other, and proving the contrapositive is sometimes easier than proving the original statement. If so, then you can proceed as follows:

1. Write, "We prove the contrapositive:" and then state the contrapositive.
2. Proceed as in a direct proof.

Example

Theorem 2.3.2. *If r is irrational, then $\sqrt{r}$ is also irrational.*

A number is *rational* when it equals a quotient of integers—that is, if it equals $\frac{m}{n}$ for some integers m and n. If it's not rational, then it's called *irrational*. So we must show that if r is *not* a ratio of integers, then $\sqrt{r}$ is also *not* a ratio of integers. That's pretty convoluted! We can eliminate both *not*'s and simplify the proof by using the contrapositive instead.

Proof. We prove the contrapositive: if $\sqrt{r}$ is rational, then r is rational. Assume that $\sqrt{r}$ is rational. Then there exist integers m and n such that:

$$\sqrt{r} = \frac{m}{n}$$

Squaring both sides gives:

$$r = \frac{m^2}{n^2}$$

Since m^2 and n^2 are integers, r is also rational. □

2.4 Proving a Biconditional

Compare with Section 1.6 from Mathematics for Computer Science

Many mathematical theorems assert that two statements are logically equivalent; that is, one holds if and only if the other does. Here is an example that has been known for several thousand years:

> Two triangles have the same side lengths if and only if two side lengths and the angle between those sides are the same.

Method 1: Prove Two Implications

By the Biconditional Law, "$P \leftrightarrow Q$" is equivalent to $(P \to Q) \wedge (Q \to P)$. This means that if both implications are true, we know the biconditional is also true, so we can prove a biconditional by proving two implications:

1. Write, "First, we show $P \to Q$." Do this by one of the methods in Section 2.3.
2. Write, "Now, we show $Q \to P$." Again, do this by one of the methods in Section 2.3.

Method 2: Chain Biconditionals Together

Another way to prove that P is true if and only if Q is true, you can prove P is equivalent to a second statement which is equivalent to a third statement and so forth until you reach Q.

Example

The standard deviation of a sequence of values $x_1, x_2, \ldots, x_n$ is defined to be:

$$\sqrt{\frac{(x_1-\mu)^2 + (x_2-\mu)^2 + \cdots + (x_n-\mu)^2}{n}} \tag{2.1}$$

where μ is the mean (average) of the values:

$$\mu := \frac{x_1 + x_2 + \cdots + x_n}{n}$$

Theorem 2.4.1. *The standard deviation of a sequence of values is zero if and only if all the values are equal to the mean.*

For example, the standard deviation of test scores is zero if and only if everyone scored exactly the class average.

Proof. We construct a chain of biconditional statements, starting with the statement that the standard deviation (2.1) is zero:

$$\sqrt{\frac{(x_1-\mu)^2+(x_2-\mu)^2+\cdots+(x_n-\mu)^2}{n}}=0 \tag{2.2}$$

Now since zero is the only number whose square root is zero, equation (2.2) holds if and only if

$$\frac{(x_1-\mu)^2+(x_2-\mu)^2+\cdots+(x_n-\mu)^2}{n}=0 \tag{2.3}$$

A fraction is equal to zero if and only if its numerator is equal to zero, so

$$(x_1-\mu)^2+(x_2-\mu)^2+\cdots+(x_n-\mu)^2=0 \tag{2.4}$$

Squares of real numbers are always nonnegative, so every term on the left-hand side of equation (2.4) is nonnegative. This means that (2.4) holds if and only if

$$\text{Every term on the left-hand side of equation (2.4) is zero.} \tag{2.5}$$

But a term $(x_i-\mu)^2$ is zero if and only if $x_i-\mu=0$, so (2.5) is true if and only if

$$\text{Every } x_i \text{ equals the mean.}$$

□

2.5 Proof by Cases

Compare with Section 1.7 from Mathematics for Computer Science

Breaking a complicated proof into cases and proving each case separately is a common, useful proof strategy. Here's an amusing example.

Let's agree that given any two people, either they have met or not. If every pair of people in a group has met, we'll call the group a *club*. If every pair of people in a group has not met, we'll call it a group of *strangers*.

Theorem 2.5.1. *Every collection of 6 people includes a club of 3 people or a group of 3 strangers.*

Proof. The proof is by case analysis. Let x denote one of the six people. There are two cases:

1. Among the 5 other people besides x, at least 3 have met x.

2. Among the 5 other people, at least 3 have not met x.

Now, we have to be sure that *at least* one of these two cases must hold at any given time. Part of a case analysis argument is showing that you've covered all possible scenarios. This is often obvious, because the two cases are of the form "P" and "$\neg P$." However, this situation is not stated quite so

simply. That being said, we've split the 5 people into two groups, those who have met x and those who have not, so one of the groups must have at least half the people.

Case 1: Suppose that at least 3 people have met x. This case splits into two subcases:

Case 1.1: No two among those people have met each other. Then these people are a group of at least 3 strangers. So the theorem holds in this subcase.

Case 1.2: There are two among those people that have met each other. Then that pair, together with x, form a club of 3 people. So the theorem holds in this subcase.

Since the theorem held in all subcases, it holds in Case 1.

Case 2: Suppose that at least 3 people have not met x. This case also splits into two subcases:

Case 2.1: All of those people have met each other. Then these people are a club of at least 3 people. So the theorem holds in this subcase.

Case 2.2: There are two among those people have not met each other. Then that pair, together with x, form a group of at least 3 strangers. So the theorem holds in this subcase.

Since the theorem held in all subcases, it holds in Case 2.

Since the theorem holds in all cases, it must be true. □

2.6 Proof by Induction

Compare with Chapter 5 from Mathematics for Computer Science[3]

Induction is a powerful method for showing a property is true for all nonnegative integers. Induction plays a central role in discrete mathematics and computer science. In fact, its use is a defining characteristic of *discrete*—as opposed to *continuous*—mathematics. This section introduces two versions of induction, Ordinary and Strong, and explains why they work and how to use them in proofs.

2.6.1 Ordinary Induction

To understand how induction works, suppose there is a professor who brings a bottomless bag of assorted miniature candy bars to her large class. She offers to share the candy in the following way. First, she lines the students up in order. Next she states two rules:

1. The student at the beginning of the line gets a candy bar.

[3]Note that in *Mathematics for Computer Science*, induction takes up an entire chapter. The authors there spend more time doing worked examples and giving deeper explanations, so if you feel like you want to understand induction better after reading this section, consider reading their chapter on it.

2. If a student gets a candy bar, then the following student in line also gets a candy bar.

Let's number the students by their order in line, starting the count with 0, as usual in computer science. Now we can understand the second rule as a short description of a whole sequence of statements:

- If student 0 gets a candy bar, then student 1 also gets one.
- If student 1 gets a candy bar, then student 2 also gets one.
- If student 2 gets a candy bar, then student 3 also gets one.

$$\vdots$$

Of course, this sequence has a more concise mathematical description:

> If student n gets a candy bar, then student $n+1$ gets a candy bar, for all nonnegative integers n.

So suppose you are student 17. By these rules, are you entitled to a miniature candy bar? Well, student 0 gets a candy bar by the first rule. Therefore, by the second rule, student 1 also gets one, which means student 2 gets one, which means student 3 gets one as well, and so on. By 17 applications of the professor's second rule, you get your candy bar! Of course the rules really guarantee a candy bar to every student, no matter how far back in line they may be.

The reasoning that led us to conclude that every student gets a candy bar is essentially all there is to induction.

The Induction Principle.

Let P be a predicate whose domain is the nonnegative integers. If

1. $P(0)$ is true
2. $P(n) \to P(n+1)$ is true for all nonnegative integers n

then

- $P(m)$ is true for all nonnegative integers m.

We call 1 the *Base Case* and 2 the *Inductive Hypothesis*.

2.6.2 Strong Induction

A useful variant of induction is called *strong induction*. Strong induction and ordinary induction are used for exactly the same thing: proving that a predicate is true for all nonnegative integers.

Strong induction is useful when a simple proof that the predicate holds for $n+1$ does not follow just from the fact that it holds at n, but from the fact that it holds for other values less than n.

The Strong Induction Principle.

Let P be a predicate whose domain is the nonnegative integers. If

1. $P(0)$ is true
2. $(P(0) \wedge P(1) \wedge P(2) \wedge \cdots \wedge P(n)) \rightarrow P(n+1)$ is true for all nonnegative integers n

then

- $P(m)$ is true for all nonnegative integers m.

The only change from the ordinary induction principle is that strong induction allows you make more assumptions in the inductive step of your proof! In an ordinary induction argument, you assume that $P(n)$ is true and try to prove that $P(n+1)$ is also true. In a strong induction argument, you may assume that $P(0)$, $P(1)$, ..., $P(n)$ are *all* true when you go to prove $P(n+1)$. So you can assume a stronger set of hypotheses which can make your job easier.

Chapter 3

Sets, Functions, and Relations

3.1 Sets

3.1.1 Set Definition

Informally, a *set* is a collection of objects, which are called *elements*. The elements of a set can be just about anything: numbers, points in space, or even other sets. The conventional way to write down a set is to list the elements inside curly-braces. For example, here are some sets:

$$\begin{array}{ll} A = \{ \text{ Alex, Tippy, Shells, Shadow } \} & \text{pets} \\ B = \{ \text{ red, blue, yellow } \} & \text{primary colors} \\ C = \{\{a, b\}, \{a, c\}, \{b, c\}\} & \text{a set of sets} \end{array}$$

This works fine for small finite sets. Other sets might be defined by indicating how to generate a list of them:

$$D = \{ \text{ 1, 2, 4, 8, 16, } \ldots \} \qquad \text{the powers of 2}$$

The order of elements is not significant, so $\{x, y\}$ and $\{y, x\}$ are the same set written two different ways. Also, any object is, or is not, an element of a given set—there is no notion of an element appearing more than once in a set.[1] So, writing $\{x, x\}$ is just indicating the same thing twice: that x is in the set. In particular, $\{x, x\} = \{x\}$.

The expression "$e \in S$" asserts that e is an element of set S. For example, letting the sets A, B, and D be as defined above, $32 \in D$ and blue $\in B$, but Tailspin $\notin A$. Sets are simple, flexible, and everywhere. You'll find some set mentioned in nearly every section of this text.

[1] It's not hard to develop a notion of *multisets* in which elements can occur more than once, but multisets are not ordinary sets and are not covered in this text.

3.1.2 Common Sets

Mathematicians have devised special symbols to represent some common sets.

Symbol	Set	Elements
$\emptyset$	The empty set	none
$\mathbb{N}$	The natural numbers	$\{ 0, 1, 2, 3, \dots \}$
$\mathbb{Z}$	The integers	$\{ \dots, -3, -2, -1, 0, 1, 2, 3, \dots \}$
$\mathbb{Q}$	The rational numbers	$\frac{1}{2}$, $-\frac{5}{3}$, $\frac{16}{1}$, etc.
$\mathbb{R}$	The real numbers	π, $\frac{4}{3}$, -9, $\sqrt{2}$, etc.
$\mathbb{C}$	The complex numbers	i, $\frac{19}{2}$, $\sqrt{2} - 2i$, etc.

Table 3.1: The most common sets in mathematics

A superscript "+" restricts a set to its positive elements; for example, $\mathbb{R}^+$ denotes the set of positive real numbers. Similarly, $\mathbb{Z}^-$ denotes the set of negative integers. Note that zero is neither positive nor negative, so 0 is not an element of any set with a superscript + or −. In particular, this means that $\mathbb{N} \neq \mathbb{Z}^+$ though the two sets are frequently used in similar contexts. Because of this, the natural numbers are sometimes called the "nonnegative integers."

3.1.3 Comparing and Combining Sets

The expression $S \subseteq T$ indicates that set S is a *subset* of set T, which means that every element of S is also an element of T. For example, $\mathbb{N} \subseteq \mathbb{Z}$ because every nonnegative integer is an integer; $\mathbb{Q} \subseteq \mathbb{R}$ because every rational number is a real number, but $\mathbb{C} \not\subseteq \mathbb{R}$ because not every complex number is a real number.

As a memory trick, think of the "$\subseteq$" symbol as like the "$\leq$" sign with the smaller set or number on the left-hand side. Notice that just as $n \leq n$ for any real number n, we also say $S \subseteq S$ for any set S.

There is also a relation $\subset$ on sets like the "less than" relation $<$ on real numbers. $S \subset T$ means that S is a subset of T, but the two are not equal. So just as $n \not< n$ for any real number n, we also say $A \not\subset A$, for every set A. "$S \subset T$" is read as "S is a proper subset of T" or "S is a strict subset of T."

There are several basic ways to combine sets.

Definition 3.1.1 (Union)**.** The *union* of sets A and B, denoted $A \cup B$, is the set which contains all the elements appearing in A or B or both. That is,

$$x \in A \cup B \equiv (x \in A \vee x \in B).$$

Definition 3.1.2 (Intersection)**.** The *intersection* of A and B, denoted $A \cap B$, consists of all

elements that appear in both A and B. That is,

$$x \in A \cap B \equiv (x \in A \wedge x \in B).$$

Definition 3.1.3 (Set Difference)**.** The *set difference*, denoted $A - B$, consists of all elements that are in A, but not in B. That is,

$$x \in A - B \equiv (x \in A \wedge x \notin B)$$

To illustrate these definitions, suppose

$$X = \{1, 2, 3\},$$
$$Y = \{2, 3, 4\}.$$

Then

$$X \cup Y = \{1, 2, 3, 4\}$$
$$X \cap Y = \{2, 3\}$$
$$X - Y = \{1\}$$

Notice that $\cup$ looks similar to $\vee$ and $\cap$ looks similar to $\wedge$. This is done on purpose to make it easier to remember which operation does what.

Often all the sets being considered are subsets of a known domain of discourse D. Then for any subset A of D, we define $\overline{A}$ to be the set of all elements of D *not* in A. That is,

$$\overline{A} = D - A.$$

The set $\overline{A}$ is called the *complement* of A in D. So

$$\overline{A} = \emptyset \leftrightarrow A = D$$

and

$$\overline{A} = D \leftrightarrow A = \emptyset.$$

For example, if the domain we're working with is the integers, the complement of the natural numbers is the set of negative integers:

$$\overline{\mathbb{N}} = \mathbb{Z}^{-}.$$

We can use complement to rephrase subset in terms of equality

$$A \subseteq B \equiv A \cap \overline{B} = \emptyset.$$

3.1.4 Power Sets

The set of all the subsets of a set A is called the *power set* of A which we denote by $\text{pow}(A)$. Applying the definition above,

$$B \in \text{pow}(A) \leftrightarrow B \subseteq A.$$

For example, the elements of $\text{pow}(\{1, 2\})$ are $\emptyset$, $\{1\}$, $\{2\}$, and $\{1, 2\}$. More generally, if A has n elements, then there are 2^n sets in $\text{pow}(A)$. For this reason, some authors use the notation 2^A to denote the power set of A.

3.1.5 Set Builder Notation

An important use of predicates is in *set builder notation*. We'll often want to talk about sets that cannot be described very well by listing the elements explicitly or by taking unions, intersections, etc., of easily described sets. Set builder notation often comes to the rescue. The idea is to define a set using a predicate; in particular, the set consists of all values that make the predicate true.

Here are some examples of set builder notation:

$$\begin{aligned}
A &= \{n \in \mathbb{N} \mid n \text{ is a prime and } n = 4k + 1 \text{ for some integer } k\},\\
B &= \{x \in \mathbb{R} \mid x^3 - 3x + 1 > 0\},\\
C &= \{a + bi \in \mathbb{C} \mid a^2 + 2b^2 \leq 1\},\\
D &= \{W \in \text{words} \mid W \text{ is used in this text}\}.
\end{aligned}$$

The set A consists of all natural numbers n for which the predicate "n is a prime and $n = 4k+1$ for some integer k" is true. Thus, the smallest elements of A are:

$$5,\ 13,\ 17,\ 29,\ 37,\ 41,\ 53,\ 61,\ 73,\ \ldots$$

Trying to indicate the set A by listing these first few elements wouldn't work very well; even after ten terms, the pattern is not obvious. Similarly, the set B consists of all real numbers x for which the predicate

$$x^3 - 3x + 1 > 0$$

is true. In this case, an explicit description of the set B in terms of intervals would require solving a cubic equation. Set C consists of all complex numbers $a + bi$ such that:

$$a^2 + 2b^2 \leq 1$$

This is an oval-shaped region around the origin in the complex plane. Finally, the members of set D can be determined by cataloging all words used in this text.

3.1.6 Proving Set Equalities

Two sets are defined to be equal if they have exactly the same elements. That is,

$$A = B \equiv \forall\, x(x \in A \leftrightarrow x \in B)$$

So, set equalities can be formulated and proved as "if and only if" theorems. For example:

Theorem 3.1.4 (Distributive Law for Sets)**.** *Let A, B, and C be sets. Then:*

$$A \cap (B \cup C) = (A \cap B) \cup (A \cap C) \tag{3.1}$$

Proof. The equality (3.1) is equivalent to the assertion that

$$z \in A \cap (B \cup C) \leftrightarrow z \in (A \cap B) \cup (A \cap C) \tag{3.2}$$

for all z. Now we'll prove (3.2) by a chain of if and only if statements.

$$\begin{aligned}
& z \in A \cap (B \cup C) && \text{(given)} \\
\leftrightarrow\ & (z \in A) \wedge (z \in B \cup C) && \text{(def of } \cap\text{)} \\
\leftrightarrow\ & (z \in A) \wedge (z \in B \vee z \in C) && \text{(def of } \cup\text{)} \\
\leftrightarrow\ & (z \in A \wedge z \in B) \vee (z \in A \wedge z \in C) && \text{(Distribution Law)} \\
\leftrightarrow\ & (z \in A \cap B) \vee (z \in A \cap C) && \text{(def of } \cap\text{)} \\
\leftrightarrow\ & z \in (A \cap B) \cup (A \cap C) && \text{(def of } \cup\text{)}
\end{aligned}$$

□

A similar argument could be given to prove that

$$A \cup (B \cap C) = (A \cup B) \cap (A \cup C).$$

Of course there are lots of ways to prove such set equality formulas, but we emphasize this chain-of-IFF's approach because it illustrates a general method. You can prove any set equality involving $\cup$, $\cap$, $-$, and complement by reducing verification of the set equality into checking validity of a corresponding propositional equivalence. We know validity can be checked in lots of ways, for example by truth table or algebra.

As a further example, from De Morgan's Laws for propositions,

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q \quad \text{and} \quad \neg(P \vee Q) \equiv \neg P \wedge \neg Q,$$

we can derive corresponding De Morgan's Laws for set equality:

$$\overline{A \cap B} = \overline{A} \cup \overline{B} \quad \text{and} \quad \overline{A \cup B} = \overline{A} \cap \overline{B}.$$

In fact, if a set equality is not valid, this approach can also be used to find a simple counter-example.

Despite this correspondence between set and propositional formulas, it's important not to confuse propositional with set operations. For example, if X and Y are sets, then it is wrong to write "$X \wedge Y$" instead of "$X \cap Y$." Applying $\wedge$ to sets will cause your compiler—or your grader—to throw a type error, because an operation that is supposed to be applied to truth values has been applied to sets. Likewise, if P and Q are propositions, then it is a type error to write "$P \cup Q$" instead of "$P \vee Q$."

3.2 Functions

3.2.1 Function Definition

A *function* assigns an element of one set, called the *domain*, to an element of another set, called the *codomain*. The notation

$$f : A \to B$$

indicates that f is a function with domain A and codomain B. The familiar notation "$f(a) = b$" indicates that f assigns the element $b \in B$ to $a \in A$. Here we would say that a *maps* to b. The set of all pairs $(a, f(a))$ is called the *graph* of f.

Functions are often defined by formulas, as in:

$$f_1 : \mathbb{R} - \{0\} \to \mathbb{R}$$

defined by

$$f_1(x) = \frac{1}{x^2},$$

or, letting D be the set of all finite strings of letters,

$$f_2 : D \times D \to D$$

defined by

$$f_2(\alpha, \beta) = \alpha\beta$$

or, letting T be the set of all finite lists of real numbers,

$$f_3 : \mathbb{R} \times \mathbb{N} \to T$$

defined by

$$f_3(x, n) = \underbrace{(x, x, x, \ldots, x)}_{n \text{ times}}.$$

A function with a finite domain could be specified by a table that shows the value of the function

at each element of the domain. For example, the function

$$f_4 : \{\mathbf{T}, \mathbf{F}\} \times \{\mathbf{T}, \mathbf{F}\} \to \{\mathbf{T}, \mathbf{F}\}$$

defined by:

P	Q	$f_4(P, Q)$
T	**T**	**T**
T	**F**	**F**
F	**T**	**T**
F	**F**	**T**

For functions with finite domains and codomains like f_4, we often draw arrow diagrams like the one in Table 3.2 below to describe how the function behaves. Each arrow connects an element of the domain to the element it maps to in the codomain.

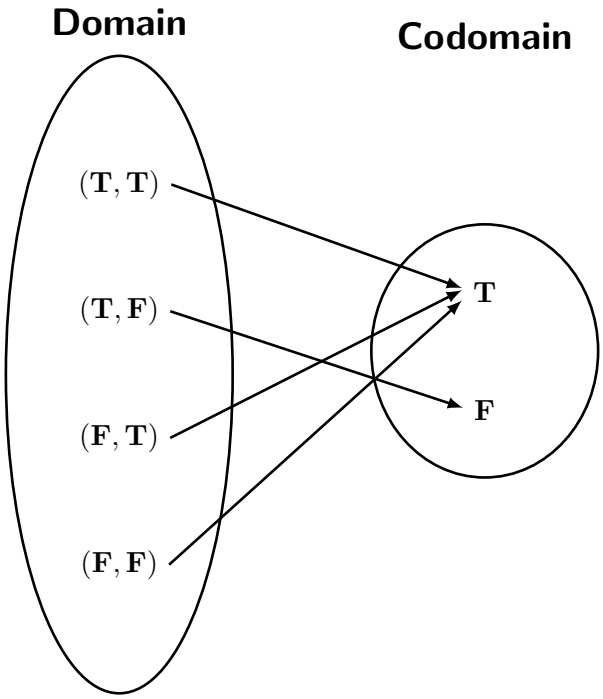


Table 3.2: f_4 as an arrow diagram

Notice that f_4 could also have been described by the formula

$$f_4(P, Q) = P \to Q.$$

A function might also be defined by a procedure for computing its value at any element of its domain, or by some other kind of specification. For example, define f_5 to return the length of a

left to right search of the bits in a binary string until a 1 appears and 0 if no 1 appears, so

$$f_5(0010) = 3$$
$$f_5(100) = 1$$
$$f_5(0000) = 0.$$

It's often useful to find the set of values a function takes when applied to the elements in a set. So if $f : A \to B$, and S is a subset of A, we define $f(S)$ to be the set of all the values that f takes when it is applied to elements of S. That is,

$$f(S) = \{b \in B \mid f(s) = b \text{ for some } s \in S\}$$

For example, if we let $[r, s]$ denote set of numbers in the closed interval from r to s on the real line, then $f_1([1, 2]) = [1/4, 1]$.

For another example, let's take the "search for a 1" function, f_5. If we let X be the set of bit strings which start with an even number of 0's followed by a 1, then $f_5(X)$ would be the positive odd integers. Given a function f and a subset S of its domain, the set $f(S)$ is referred to as the *image* of S under f. The set of values that arise from applying f to all elements of its domain is called the *range* of f. That is,

$$\text{range}(f) = f(\text{domain}(f)).$$

Some authors refer to the codomain as the range of a function, but they shouldn't. The codomain is the set of all *possible* values a function may output, while the range is the set of all *actual* values the function does output. For example, the function $s : \mathbb{R} \to \mathbb{R}$ defined by $s(x) = x^2$ and the function $t : \mathbb{R} \to \mathbb{C}$ defined by $t(x) = x^2$ have the same range but different codomains. The distinction between the range and the codomain will be important when we discuss properties of functions.

3.2.2 Function Composition

Doing things step by step is a universal idea. Taking a walk is a literal example, but so is cooking from a recipe, executing a computer program, evaluating a formula, and recovering from substance abuse.

Abstractly, taking a step amounts to applying a function, and going step by step corresponds to applying functions one after the other. This is captured by the operation of *composing* functions. Composing the functions f and g means that first f is applied to some argument to produce $f(x)$, and then g is applied to that result to produce $g(f(x))$.

Definition 3.2.1. For functions $f : A \to B$ and $g : C \to D$ where $\text{range}(f) \subseteq C$, the *composition* of g with f is the function $g \circ f : A \to D$ defined by

$$(g \circ f)(x) = g(f(x))$$

for all $x \in A$.

Note the restriction that the range of f must be a subset of the domain of g.[2] If this were not the case, then g would not know what to do with the output of f! This added restriction ensures that plugging $f(x)$ into g actually makes sense.

3.2.3 Properties of Functions

There are a few special properties of functions that we care about. For the remainder of this section we will let f be an arbitrary function with domain A and codomain B.

Definition 3.2.2 (Well-defined)**.** A function is *well-defined* if writing the input in a different way always gives the same output. In other words,

$$\forall\, x, y \in A\, (x = y \to f(x) = f(y))$$

Consider the following example. Let $a : \mathbb{Q} \to \mathbb{Z}$ be the function defined by

$$a\left(\frac{n}{d}\right) = n + d.$$

Then $a\left(\frac{1}{2}\right) = 1 + 2 = 3$, but $a\left(\frac{2}{4}\right) = 2 + 4 = 6$. This function is not well-defined because $\frac{1}{2} = \frac{2}{4}$ but $a\left(\frac{1}{2}\right) \neq a\left(\frac{2}{4}\right)$. As another example, let N_2 be the set of sets that contain two natural numbers. So $\{0, 9\} \in N_2$, $\{100, 5\} \in N_2$, etc. Let $b : N_2 \to \mathbb{Z}$ be the function defined by

$$b(\{x, y\}) = x - y.$$

Then $b(\{0, 1\}) = 0 - 1 = -1$ and $b(\{1, 0\}) = 1 - 0 = 1$. b is not well-defined because $\{0, 1\} = \{1, 0\}$, but $b(\{0, 1\}) \neq b(\{1, 0\})$.

Any time a function has a domain whose elements can be written in multiple ways, you have to be very careful that to make sure the function is well-defined.

Definition 3.2.3 (Injective)**.** A function is *injective* or *one-to-one* if no two distinct elements of the domain map to the same element of the codomain. In other words,

$$\forall\, x, y \in A\, (x \neq y \to f(x) \neq f(y)).$$

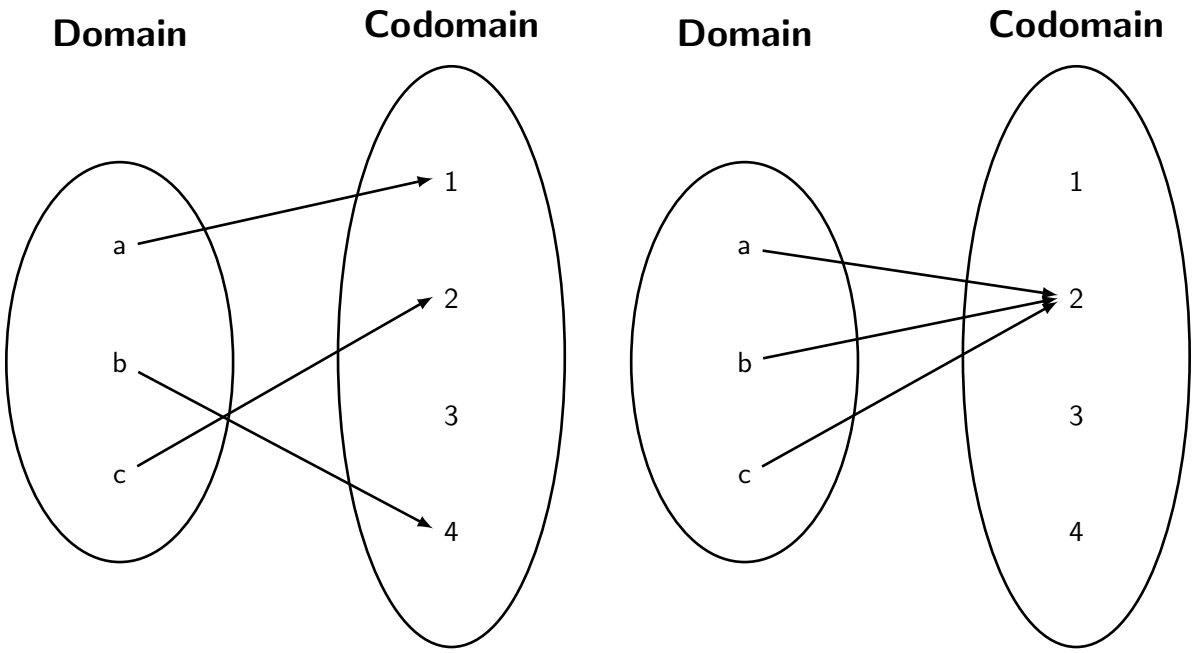
An injective function is called an *injection*.

More often than not when working with injectivity in propositional logic and/or proofs, we use the contrapositive because equality is easier to work with than inequality:

$$\forall\, x, y \in A\, (f(x) = f(y) \to x = y).$$

[2] In practice, it is usually easier to check that the codomain of f is a subset (or even equal to) the domain of g to avoid having to compute the range. That being said, checking the range allows more functions to be composed.

In terms of an arrow diagram, being injective means that every element of the codomain has *at most* one arrow pointing to it. This is illustrated by Table 3.3 below. Notice how in the left diagram, every element of the codomain has at most one arrow pointing to it while in the right diagram, the element 2 has more than one arrow.



Injective Not injective

Table 3.3: Visual example of an injective function (left) and a function that is not injective (right)

As another example, the function $c : \mathbb{R} \to \mathbb{R}$ defined by

$$c(x) = x^2$$

is not injective because both -1 and 1 map to the same output, i.e. $c(-1) = c(1)$. In this example, you could have picked any nonzero real number and its negative as your counterexample, but you only need one counterexample to prove the function is not injective.

Definition 3.2.4 (Surjective)**.** A function is *surjective* or *onto* if every element of the codomain is mapped to by some element of the domain. In other words,

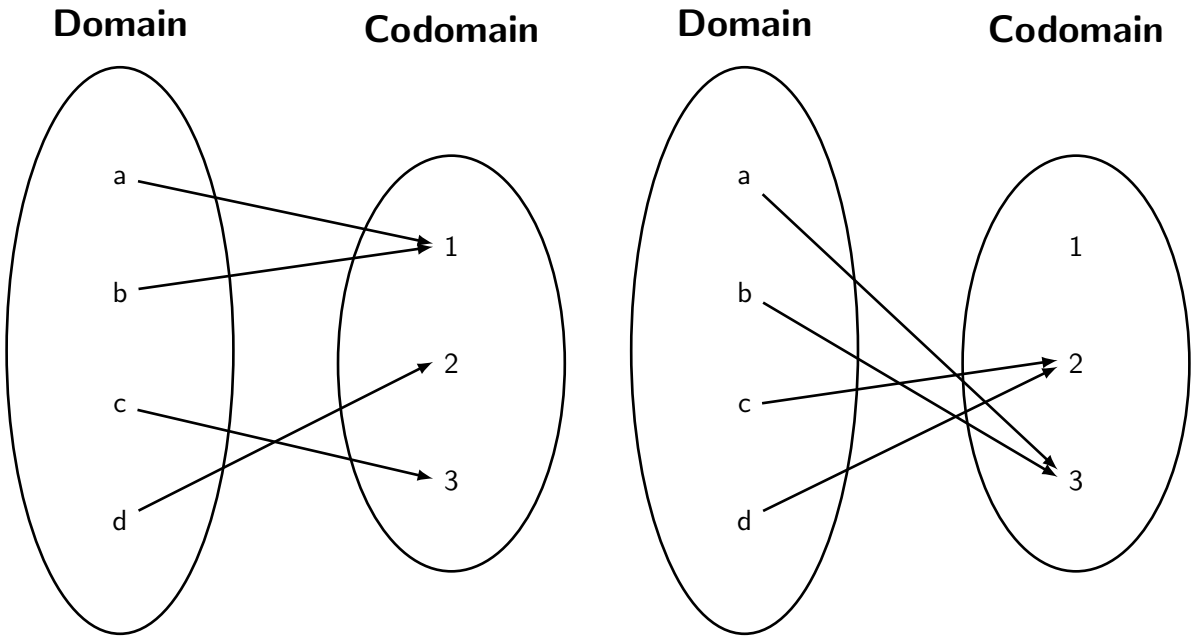
$$\forall\, y \in B\, \exists\, x \in A\, (f(x) = y).$$

A surjective function is called a *surjection*.

An equivalent condition for a function to be surjective is if the range of the function is equal to the codomain.

In terms of an arrow diagram, being surjective means that every element of the codomain has *at least* one arrow pointing to it. This is illustrated by Table 3.4 below. Notice how in the left

diagram, every element of the codomain has at least one arrow pointing to it while in the right diagram, the element 1 has none.



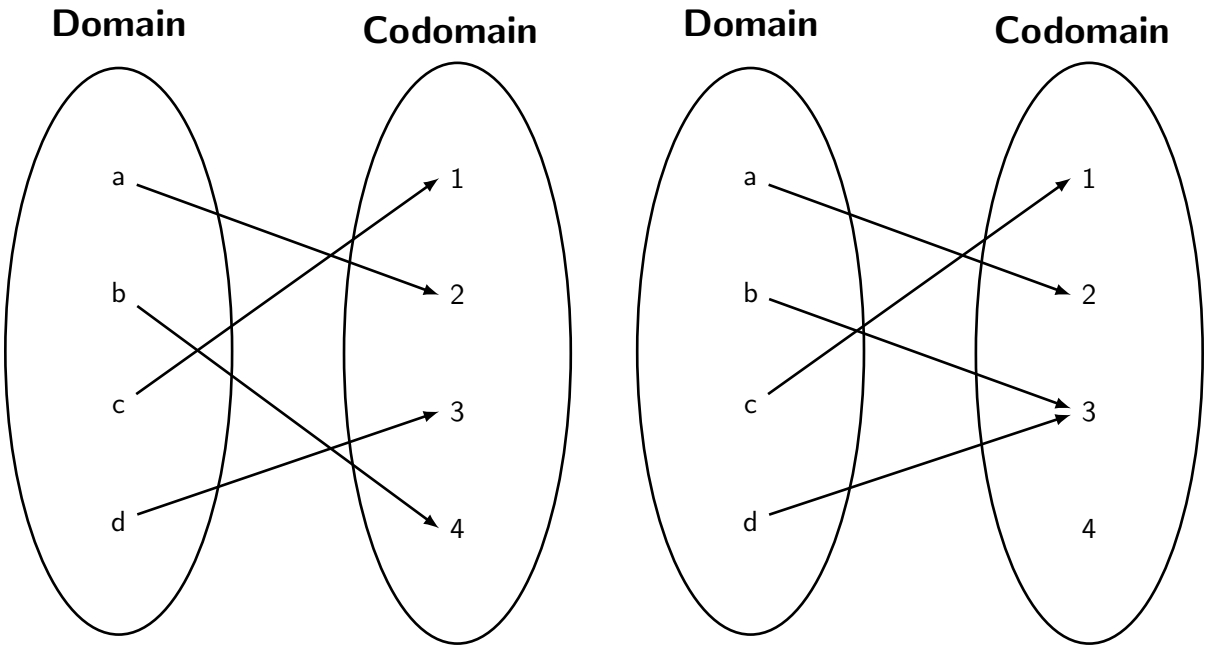
Surjective Not surjective

Table 3.4: Visual example of a surjective function (left) and a function that is not surjective (right)

Going back to the function $c : \mathbb{R} \to \mathbb{R}$ defined before, we see that it is not surjective in addition to not being injective because no input maps to -1.

Definition 3.2.5 (Bijective)**.** We say a function is *bijective* if it is both injective and surjective. A bijective function is called a *bijection*.

In terms of an arrow diagram, being bijective means that every element of the codomain has *exactly* one arrow pointing to it. Remember, injective means the elements of the codomain have *at most* one arrow and surjective means they have *at least* one arrow. This is illustrated by Table 3.5 below. Notice how in the left diagram, every element of the codomain has at exactly arrow pointing to it while in the right diagram, the element 3 has two arrows and the element 4 has none.

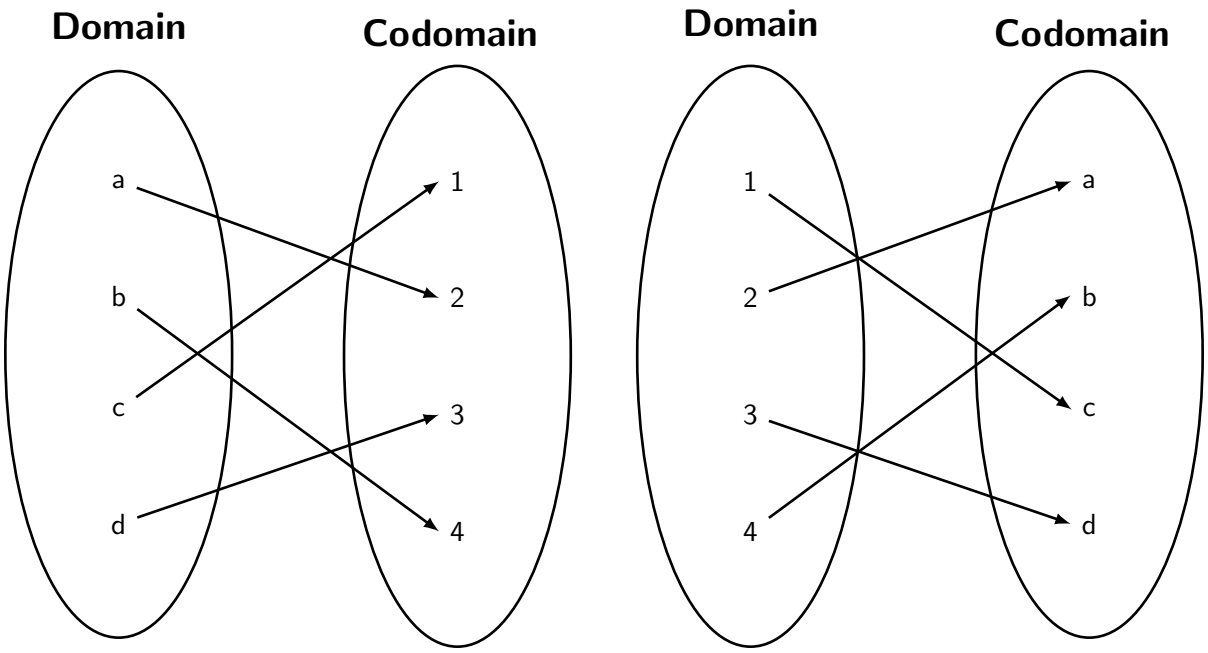


Bijective | Not bijective

Table 3.5: Visual example of a bijective function (left) and a function that is not bijective (right)

Properties of Bijections

Bijections have the unique property that, given any bijective function, we can flip the arrows to get a new function. This special related function is called the *inverse* of the original function and is written with a superscript -1, so the inverse of $f : A \to B$ would be written as $f^{-1} : B \to A$. Since bijections have inverses, we say that bijections are *invertible*.



Original function Inverse function

Table 3.6: Visual example of a bijective function (left) and its inverse (right)

If you compose a function with its inverse, you get what is called the *identity function* on either the domain or the codomain (depending on the order of composition). The identity function always maps a set into itself by taking each element and mapping it to itself. So the identity function on $\mathbb{N}$ would map 0 to 0, map 1 to 1, map 2 to 2, etc. The identity function on a set S is often written as id_S or even just id if the set being acted on is clear from context. Given a bijection $f : A \to B$ and its inverse $f^{-1} : B \to A$, we would have $f^{-1} \circ f = id_A$ and $f \circ f^{-1} = id_B$.

If you are given two functions f and g and need to check if they are inverses, the fastest way to do so is to compute $f \circ g$ and $g \circ f$ (you must check both!) to see if you get the identity. If so, then the functions are inverses, otherwise they are not.

For example, let f and g be the real-valued functions $f(x) = e^x$ and $g(x) = \ln(x)$. We compute

$$(f \circ g)(x) = e^{\ln(x)} = x$$

and

$$(g \circ f)(x) = \ln(e^x) = x.$$

In both cases, we input x and get x as the output, so the functions are inverses.

As another example, consider the function $h(x) = \frac{1}{x}$. Is it its own inverse? To check, we compute

$$(h \circ h)(x) = \frac{1}{\frac{1}{x}} = x,$$

so it seems like it should be, right? We have to be careful here because the answer depends on the

codomain. If the codomain of h is $\mathbb{R}$, then h is not invertible (where would the inverse function send 0?). If the codomain of h is $\mathbb{R} - \{0\}$ like its domain, then h is in fact its own inverse.

Bijections are particularly important when discussing set cardinality. For finite sets, the *cardinality* of a set is the number of distinct elements the set contains, so the set $A = \{1, 10, 100, 1000\}$ would have cardinality 4. We use a notation similar to absolute value to denote cardinality, so in the above example, we would write $|A| = 4$. Since a bijection has exactly one arrow for every element of the domain (definition of a function) and exactly one arrow for every element of the codomain (injective & surjective), the domain and codomain of a bijective function must always have the same number of elements. This means that if we want to prove that two finite sets have the same number of elements, it is sufficient to construct a bijection between them.

Theorem 3.2.6. *For any two finite sets A and B, $|A| = |B|$ if and only if there exists a bijection between them.*

Since you can't count the elements of an infinite set, this is how we define cardinality for infinite sets. We say two infinite sets have the same cardinality if there exists a bijection between them. For example, under this definition, $\mathbb{N}$, $\mathbb{Z}$, and $\mathbb{Q}$ all have the same cardinality, but $\mathbb{R}$ has a different cardinality. This is an important fact in mathematics, though we include it here only as a motivation for why we would use bijections to measure the size of sets rather than simply counting the elements.

3.3 Binary Relations

3.3.1 Binary Relation Definition

Binary relations define relations between two objects. For example, "less-than" on the real numbers relates every real number a to a real number b, precisely when $a < b$. Similarly, the subset relation relates a set A to another set B precisely when $A \subseteq B$. A function $f : A \to B$ is a special case of binary relation in which an element $a \in A$ is related to an element $b \in B$ precisely when $f(a) = b$.

In this section we'll define some basic vocabulary and properties of binary relations.

Definition 3.3.1. A *binary relation* R consists of a set A, called the *domain* of R, a set B called the *codomain* of R, and a subset of $A \times B$ called the *graph* of R.

A relation whose domain is A and codomain is B is said to be "between A and B", or "from A to B." As with functions, we write $R : A \to B$ to indicate that R is a relation from A to B. When the domain and codomain are the same set A we simply say the relation is "on A." It's common to use "aRb" to mean that the pair (a, b) is in the graph of R. For each pair (a, b) in the graph of R, we say that "a is related to b" or "a relates to b" under R.

Notice that the definition is exactly the same as the definition of a function, except that a function must map each element of the domain to exactly one element of the codomain while a a relation can relate each element of the domain to zero or more elements of the codomain. In other

words, the graph of a relation can contain any number of pairs of the form $(a, \text{something})$. As we said, a function is a special case of a binary relation.

The "in-charge of" relation `Chrg` for MIT in Spring 2010 subjects and instructors is a handy example of a binary relation. Its domain `Fac` is the names of all the MIT faculty and instructional staff, and its codomain is the set `SubNums` of subject numbers in the Fall 2009–Spring 2010 MIT subject listing. The graph of `Chrg` contains precisely the pairs of the form

([instructor-name], [subject-num])

such that the faculty member named [instructor-name] is in charge of the subject with number [subject-num] that was offered in Spring 2010. So graph(`Chrg`) contains pairs like:

(T. Eng, 6.UAT)
(G. Freeman, 6.011)
(G. Freeman, 6.UAT)
(G. Freeman, 6.881)
(G. Freeman, 6.882)
(J. Guttag, 6.00)
(A. R. Meyer, 6.042)
(A. R. Meyer, 18.062)
(A. R. Meyer, 6.844)
(T. Leighton, 6.042)
(T. Leighton, 18.062)
⋮

Some subjects in the codomain `SubNums` do not appear among this list of pairs—that is, they are not in range(`Chrg`). These are the Fall term-only subjects. Similarly, there are instructors in the domain `Fac` who do not appear in the list because they are not in charge of any Spring term subjects. Both the domain and codomain have elements that appear multiple times in the graph. Relations place no restrictions on how the elements of the domain and codomain are related.

3.3.2 Relational Properties

Just like with functions, relations can be either well-defined or not. Because of this, one has to be careful defining a relation on sets with multiple representatives for each element to make sure that the relation is well-defined. In addition to well-defined, there are other properties of relations that we care about.

Definition 3.3.2 (Reflexive)**.** We say a binary relation $R : A \to A$ is *reflexive* if every element is related to itself. That is,

$$\forall\, a \in A\, (aRa).$$

As an example of reflexivity, consider the relation on the set $\{1, 2, 3, 4\}$ whose graph is:

$$\{(1,1), (1,2), (2,1), (2,2), (2,4), (3,1), (3,3), (4,1), (4,2), (4,3), (4,4)\}.$$

Since the domain is $\{1, 2, 3, 4\}$, we we only need to check if $(1, 1)$, $(2, 2)$, $(3, 3)$, and $(4, 4)$ are in the graph. Since they are, this relation is reflexive.

Now consider the relation on $\{1, 2, 3, 4, 5\}$ with the same graph as before. This new relation is not reflexive because it is missing $(5, 5)$.

Of the properties we discuss in this section, reflexivity is the only property that is not stated as an implication. This means that it cannot be vacuously true. This guarantees (as long as your set A is not the empty set) that the graph must contain some pairs—namely the graph must contain the pair (a, a) for every element $a \in A$.

Definition 3.3.3 (Symmetric)**.** We say a binary relation $R : A \to A$ is *symmetric* if, given $a, b \in A$, we get that b relates to a whenever a relates to b. That is,

$$\forall\, a, b \in A\, (aRb \to bRa).$$

As an example of symmetry, consider the relation on the set $\{1, 2, 3, 4\}$ whose graph is:

$$\{(1,1), (1,2), (1,3), (2,1), (2,2), (2,3), (3,1), (3,2).\}$$

To check if this relation is symmetric, we need to go through each pair in the graph and see if the symmetric pair is also in the graph. Working from left to right, we get Table 3.7 below.

Pair	Symmetric Pair	Is Symmetric Pair in the Graph?
$(1,1)$	$(1,1)$	Yes
$(1,2)$	$(2,1)$	Yes
$(2,1)$	$(1,2)$	Yes
$(1,3)$	$(3,1)$	Yes
$\vdots$	$\vdots$	$\vdots$
$(3,2)$	$(2,3)$	Yes

Table 3.7

Since the symmetric pair of each pair in the graph is also in the graph, this relation is symmetric. Notice how the element 4 from the domain was not included in any pair in the graph. That is fine since we only care about the pairs that are already in the graph when checking for symmetry.

Now consider the relation on the set $\{1, 2, 3, 4\}$ whose graph is:

$$\{(1,1), (1,2), (2,1), (2,3), (3,1), (3,2), (4,4).\}$$

This relation is not symmetric because the pair $(3, 1)$ is in the graph, but the pair $(1, 3)$ is not.

Reflexivity and symmetry are often confused because of their similar names. One way to remember the difference is to think of a mirror. When a mirror *reflects* something, you get the *same thing* back. So reflexivity means that the relation must have all things related to themselves, while symmetry means that you can switch the order of the elements in any valid relation and get another valid relation.

Definition 3.3.4 (Transitive)**.** We say a binary relation $R : A \to A$ is *transitive* if, given $a, b, c \in A$ with aRb and bRc, we also have that aRc. That is,

$$\forall\, a, b, c \in A\, ((aRb \wedge bRc) \to aRc).$$

As an example of transitivity, consider the relation $<$ on the real numbers. If $a < b$ and $b < c$, then it must be the case that $a < c$. As another example, consider the relation on $\{1, 2, 3\}$ whose graph is:

$$\{(1, 1), (1, 2), (2, 1), (2, 2), (3, 1), (3, 2)\}.$$

To check if this relation is transitive, we must compare each pair (a, b) with all pairs of the form (b, c) to see if (a, c) is in the graph. We do this in Table 3.8 below.

Pair 1	Pair 2	Resultant Pair	Is Resultant Pair in the Graph?
$(1, 1)$	$(1, 1)$	$(1, 1)$	Yes
	$(1, 2)$	$(1, 2)$	Yes
$(1, 2)$	$(2, 1)$	$(1, 1)$	Yes
	$(2, 2)$	$(1, 2)$	Yes
$(2, 1)$	$(1, 1)$	$(2, 1)$	Yes
	$(1, 2)$	$(2, 2)$	Yes
$(2, 2)$	$(2, 1)$	$(2, 1)$	Yes
	$(2, 2)$	$(2, 2)$	Yes
$(3, 1)$	$(1, 1)$	$(3, 1)$	Yes
	$(1, 2)$	$(3, 2)$	Yes
$(3, 2)$	$(2, 1)$	$(3, 1)$	Yes
	$(2, 2)$	$(3, 2)$	Yes

Table 3.8

Checking whether or not a relation is transitive can be *very* time consuming for relations with many pairs in their graph.

Now consider the relation on the set $\{1, 2, 3, 4\}$ whose graph is:

$$\{(1, 1), (1, 2), (2, 1), (2, 2), (3, 1), (3, 2), (4, 1)\}.$$

This relation is not transitive because its graph contains the pairs $(4, 1)$ and $(1, 2)$, but not $(4, 2)$.

Definition 3.3.5 (Antisymmetric)**.** We say a binary relation $R : A \to A$ is *antisymmetric* if, given $a, b \in A$ with aRb and bRa, we also have that $a = b$. That is,

$$\forall\, a, b \in A\, ((aRb \wedge bRa) \to a = b).$$

As an example of antisymmetry, consider the relation $\leq$ on the real numbers. If it is the case that $a \leq b$ and $b \leq a$, we must have that $a = b$. For similar reasons, the relation $\subseteq$ on sets is antisymmetric. Consider now the relation $<$ on the real numbers. This relation is vacuously antisymmetric because it can never be the case that both $a < b$ and $b < a$ are true simultaneously.

Be careful to note that *antisymmetric* is different from *not symmetric*. Because of this, it is actually possible for a binary relation to be both symmetric and antisymmetric at the same time. In order to do so, however, the only elements that can be related are an element with itself.

Take, for example, the relation $R : \mathbb{R} \to \mathbb{R}$ whose graph is $\{(x, x) \mid x \in \mathbb{Z}\}$. We get that R is symmetric because the only pairs are of the form (x, x), which are their own symmetric pair, and R is antisymmetric because any two pairs (a, b) and (b, a) are really just both (a, a), so $b = a$. Notice that this relation is not reflexive—we are missing the reflexive pairs for any real number that is not an integer, such as $(0.5, 0.5)$.

Equivalence Relations

A relation that is reflexive, symmetric, and transitive is called an *equivalence relation*. Equivalence relations partition a set into disjoint subsets called *equivalence classes* where any two elements of an equivalence class are related and no two elements in different equivalence classes are related.

For example, let $P : \mathbb{Z} \to \mathbb{Z}$ be the relation where, given $a, b \in \mathbb{Z}$, we have that aPb precisely when $a + b$ is even. We show that P is reflexive, symmetric, and transitive.

(reflexive) P is reflexive because $a + a = 2a$ is even for any integer a.

(symmetric) P is symmetric because $a + b = b + a$ for any integers a and b.

(transitive) Note that "$a + b$ is even" means that either a and b are both odd or both even. This means that if aPb and bPc, then a, b, and c must be either all odd or all even. This gives us that aPc, so P is transitive.

This equivalence relation is called *parity*, and the two equivalence classes we get from P are the even integers and the odd integers. We would say that any two even (or odd) integers are *equivalent* under parity, or in the specific case of the parity equivalence relation, that they share the same parity.

Equivalence classes are important because they allow us to preserve only the properties we care about. For example, if you want to know if a number is even or not, you are really asking whether or not the number is in the evens equivalence class under parity.

As another example, consider the relation on the set $\{(x, y) \in \mathbb{Z}^2 \mid y \neq 0\}$ where two pairs (a, b) and (c, d) are related whenever $ad = bc$. For example, $(1, 2)$ would be related to $(2, 4)$ because $1 \times 4 = 2 \times 2$. This is an equivalence relation. If we instead write each element (x, y) in the form $\frac{x}{y}$, we see that this relation is exactly the definition of equality in $\mathbb{Q}$. Because of this, we can think of rational numbers as being equivalence classes.

Chapter 4

Graph Theory

4.1 Digraphs

Compare with Chapter 10 from Mathematics for Computer Science

Directed graphs, called *digraphs* for short, provide a handy way to represent how things are connected together and how to get from one thing to another by following those connections. They are usually pictured as a bunch of dots or circles with arrows between some of the dots, as in Figure 10.1. The dots are called *nodes* or *vertices* and the lines are called *directed edges* or *arrows*; the digraph in Figure 10.1 has 4 nodes and 6 directed edges.

Digraphs appear everywhere in computer science. For example, the digraph in Figure 10.2 represents a communication net, a topic we'll explore in depth in Chapter 11. Figure 10.2 has three "in" nodes (pictured as little squares) representing locations where packets may arrive at the net, the three "out" nodes representing destination locations for packets, and the remaining six nodes (pictured with little circles) represent switches. The 16 edges indicate paths that packets can take through the router.

Another place digraphs emerge in computer science is in the hyperlink structure of the World Wide Web. Letting the vertices $x_1, \ldots, x_n$ correspond to web pages, and using arrows to indicate when one page has a hyperlink to another, results in a digraph like the one in Figure 10.3—although the graph of the real World Wide Web would have n be a number in the billions and probably even the trillions. At first glance, this graph wouldn't seem to be very interesting. But in 1995, two students at Stanford, Larry Page and Sergey Brin, ultimately became multibillionaires from the realization of how useful the structure of this graph could be in building a search engine. So pay attention to graph theory, and who knows what might happen!

Definition 4.1.1. A *directed graph* G consists of a nonempty set $V(G)$, called the *vertices* or *nodes* of G, and a set $E(G)$, called the *edges* of G. Each edge in $E(G)$ *starts* at some vertex u called the *tail* of the edge, and ends at some vertex v called the *head* of the edge. Such an edge can be represented by the ordered pair (u, v).

For an edge e with tail u and head v, we define $\texttt{tail}(e) = u$ and $\texttt{head}(e) = v$.

There is nothing new in Definition 4.1.1 except for a lot of vocabulary. Formally, a digraph G is the same as a binary relation on the set $V(G)$—that is, a digraph is just a binary relation whose domain and codomain are the same set $V(G)$. In fact, we've already referred to the arrows in a relation G as the "graph" of G.

4.1.1 Vertex Degrees

Compare with Section 10.1 from Mathematics for Computer Science

The *in-degree* of a vertex in a digraph is the number of arrows coming into it, and similarly its *out-degree* is the number of arrows out of it. More precisely,

Definition 4.1.2. If G is a digraph and $v \in V(G)$, then

$$\texttt{indeg}(v) = |\{e \in E(G) : \texttt{head}(e) = v\}|$$

$$\texttt{outdeg}(v) = |\{e \in E(G) : \texttt{tail}(e) = v\}|$$

An immediate consequence of this definition is

Lemma 4.1.3.

$$\sum_{v \in V(G)} \mathit{indeg}(v) = |E(G)| = \sum_{v \in V(G)} \mathit{outdeg}(v).$$

Proof. The sum of the in-degrees is the same as counting up all the heads of edges in $E(G)$. Since every edge has exactly one head, this comes out to be $|E(G)|$, the number of edges in G. Similarly, the sum of the out-degrees is the number of tails which would also be $|E(G)|$. □

4.2 Undirected Graphs

Compare with Chapter 12 from Mathematics for Computer Science

Undirected graphs model relationships that are symmetric, meaning that the relationship is mutual. Examples of such mutual relationships are being married, speaking the same language, not speaking the same language, occurring during overlapping time intervals, or being connected by a conducting wire. They come up in all sorts of applications, including scheduling, constraint satisfaction, computer graphics, and communications.

4.2.1 Vertex Adjacency and Degrees

Compare with Section 12.1 from Mathematics for Computer Science

Undirected graphs are defined in almost the same way as digraphs, except that edges are *undirected*—they connect two vertices without pointing in either direction between the vertices. So instead of a directed edge which starts at vertex v and ends at vertex w, a simple graph only has an undirected edge that connects v and w. It is common to model undirected graphs as *symmetric*

digraphs where every edge (v, w) implies the existence of the symmetric edge (w, v), though we will take a different approach here.

Definition 4.2.1. An *undirected graph* G consists of a nonempty set, $V(G)$, called the *vertices* or *nodes* of G, and a set $E(G)$ called the *edges* of G. Each edge in $E(G)$ has two vertices $u, v \in V(G)$ called its *endpoints*. Such an edge can be represented by the two element set $\{u, v\}$.

Figure 12.1 An example of a graph with 9 vertices and 8 edges.

For example, let H be the graph pictured in Figure 12.1. The vertices of H correspond to the nine dots in Figure 12.1, that is,

$$V(H) = \{a, b, c, d, e, f, g, h, i\}.$$

The edges correspond to the eight lines, that is,

$$E(H) = \{\{a, b\}, \{a, c\}, \{b, d\}, \{c, d\}, \{c, e\}, \{e, f\}, \{e, g\}, \{h, i\}\}.$$

Mathematically, that's all there is to the graph H.

Definition 4.2.2. Two vertices in an undirected graph are said to be *adjacent* if they are the endpoints of the same edge, and an edge is said to be *incident* to each of its endpoints. The number of edges incident to a vertex v is called the *degree* of the vertex and is denoted by $\texttt{deg}(v)$. Equivalently, the degree of a vertex is the number of vertices adjacent to it.

For example, for the graph H of Figure 12.1, vertex a is adjacent to vertex b, and b is adjacent to d. The edge $\{a, c\}$ is incident to its endpoints a and c. Vertex h has degree 1, d has degree 2, and $\texttt{deg}(e) = 3$. It is possible for a vertex to have degree 0, in which case it is not adjacent to any other vertices. An undirected graph G does not need to have any edges at all. $|E(G)|$ could be zero, implying that the degree of every vertex would also be zero.

A *simple graph* is a special case of an undirected graph. In a simple graph there cannot be more than one edge between the same two vertices. Also self-loops—edges that begin and at the same vertex—are not allowed in simple graphs.

There is a simple connection between the sum of the degrees and the number of edges in any graph:

Lemma 4.2.3 (Handshaking Lemma)**.** *The sum of the degrees of the vertices in a graph equals twice the number of edges, that is*

$$\sum_{v \in V(G)} \texttt{deg}(v) = 2|E(G)|.$$

Proof. Every edge contributes two to the sum of the degrees, one for each of its endpoints. □

We refer to Lemma 4.2.3 as the *Handshaking Lemma*: if we total up the number of people each person at a party shakes hands with, the total will be twice the number of handshakes that occurred.

Chapter 5

Number Theory

5.1 Divisibility

Compare with Section 9.1 from Mathematics for Computer Science

The nature of number theory emerges as soon as we consider the *divides* relation on the integers.

Definition 5.1.1. For two integers a and b, we say that a divides b (notation $a \mid b$) if there is an integer k such that $ak = b$.

The divides relation comes up so frequently that multiple synonyms for it are used all the time. The following phrases all say the same thing:

- $a \mid b$,
- a divides b,
- a is a divisor of b,
- a is a factor of b,
- b is divisible by a,
- b is a multiple of a.

Some immediate consequences of Definition 5.1.1 are that for all n,

$$n \mid 0 \qquad n \mid n, \text{ and} \qquad \pm 1 \mid n$$

Also,

$$0 \mid n \to n = 0.$$

Dividing seems simple enough, but let's play with this definition. The Pythagoreans, an ancient sect of mathematical mystics, said that a number is *perfect* if it equals the sum of its positive integral divisors, excluding itself. For example, $6 = 1+2+3$ and $28 = 1+2+4+7+14$ are perfect numbers.

On the other hand, 10 is not perfect because $1 + 2 + 5 = 8 \neq 10$, and 12 is not perfect because $1 + 2 + 3 + 4 + 6 = 16 \neq 12$. Euclid characterized all the even perfect numbers around 300 BC. But is there an *odd* perfect number? More than two thousand years later, we still don't know! All numbers up to about 10^{300} have been ruled out, but no one has proved that there isn't an odd perfect number waiting just over the horizon.

So a half-page into number theory, we've strayed past the outer limits of human knowledge. This is pretty typical; number theory is full of questions that are easy to pose, but incredibly difficult to answer.

5.1.1 Facts about Divisibility

The following lemma collects some basic facts about divisibility.

Lemma 5.1.2.

1. *If $a \mid b$ and $b \mid c$, then $a \mid c$.*
2. *If $a \mid b$ and $a \mid c$, then $a \mid sb + tc$ for all integers s and t.*
3. *For all $c \neq 0$, $a \mid b$ if and only if $ca \mid cb$.*

Proof. These facts all follow directly from Definition 5.1.1. To illustrate this, we'll prove just part 2:

Given that $a \mid b$, there is some $k_1 \in \mathbb{Z}$ such that $ak_1 = b$. Likewise, there exists some $k_2 \in \mathbb{Z}$ such that $ak_2 = c$, so

$$sb + tc = s(k_1 a) + t(k_2 a) = (sk_1 + tk_2)a.$$

Therefore $sb + tc = k_3 a$ where $k_3 = sk_1 + tk_2$, which means that

$$a \mid sb + tc.$$

□

5.1.2 When Divisibility Goes Bad

As you learned in elementary school, if one number does not evenly divide another, you get a "quotient" and a "remainder" left over. More precisely:

Theorem 5.1.3 (Division Algorithm)**.** *Let n and d be integers such that $d \neq 0$. Then there exists a unique pair of integers q and r, such that*

$$n = q \cdot d + r \text{ and } 0 \leq r < |d|.$$

The number q is called the quotient and the number r is called the remainder of n divided by d. We use the notation $\texttt{qnt}(n, d)$ for the quotient and $\texttt{rem}(n, d)$ for the remainder. Despite being

called the "Division Algorithm," Theorem 5.1.3 does not actually describe a division procedure for computing the quotient and remainder.

The absolute value notation $|d|$ used above is probably familiar from introductory calculus, but for the record, let's define it.

Definition 5.1.4. For any real number r, the absolute value $|r|$ of r is:

$$|r| = \begin{cases} r & \text{if } r \geq 0, \\ -r & \text{if } r < 0. \end{cases}$$

So by definition, the *remainder* $\texttt{rem}(n, d)$ is *nonnegative* regardless of the sign of n and d. For example, $\texttt{rem}(-11, 7) = 3$, since $11 = -2 \cdot 7 + 3$. Because of the constraint that $0 \leq r < |d|$, it would be incorrect to say that $\texttt{rem}(-11, 7) = -4$, despite the fact that $11 = -1 \cdot 7 + (-4)$.

"Remainder" operations built into many programming languages can be a source of confusion. For example, the expression "32 % 5" will be familiar to programmers in Java, C, and C++; it evaluates to $\texttt{rem}(32, 5) = 2$ in all three languages. On the other hand, these and other languages are inconsistent in how they treat remainders like "32 % -5" or "-32 % 5" that involve negative numbers. So don't be distracted by your familiar programming language's behavior on remainders, and stick to the mathematical convention that *remainders are nonnegative*.

5.2 The Greatest Common Divisor

Compare with Section 9.2 from Mathematics for Computer Science

A *common divisor* of a and b is a number that divides them both. The *greatest common divisor* of a and b is written $\texttt{gcd}(a, b)$. For example, $\texttt{gcd}(18, 24) = 6$.

As long as a and b are not both 0, they will have a gcd. The gcd turns out to be very valuable for reasoning about the relationship between a and b and for reasoning about integers in general. We'll be making lots of use of gcd's in what follows.

Some immediate consequences of the definition of gcd are that

$$\begin{aligned} &\texttt{gcd}(n, 1) = 1, \\ &\texttt{gcd}(n, n) = \texttt{gcd}(n, 0) = |n| \qquad \text{for } n \neq 0, \end{aligned}$$

where the last equality follows from the fact that everything is a divisor of 0.

5.2.1 Euclid's Algorithm

The first thing to figure out is how to find gcd's. A good way called *Euclid's algorithm* has been known for several thousand years. It is based on the following elementary observation.

Lemma 5.2.1 (The Euclidean Algorithm)**.** *For $b \neq 0$,*

$$\texttt{gcd}(a, b) = \texttt{gcd}(b, \texttt{rem}(a, b)).$$

Proof. By the Division Algorithm, Theorem 5.1.3,

$$a = qb + r \tag{5.1}$$

where $r = \texttt{rem}(a, b)$. So any divisor of b and r is a divisor of a by Lemma 5.1.2. Rearranging (5.1) gives us

$$r = a - qb,$$

so any divisor of a and b is a divisor of r by the same lemma. This means that a and b have the same common divisors as b and r, and so they have the same greatest common divisor. □

Lemma 5.2.1 is useful for quickly computing the greatest common divisor of two numbers. For example, we could compute the greatest common divisor of 1147 and 899 by repeatedly applying it:

$$\begin{aligned}
\texttt{gcd}(1147, 899) &= \texttt{gcd}(899, \texttt{rem}(1147, 899) = 248) \\
&= \texttt{gcd}(248, \texttt{rem}(899, 248) = 155) \\
&= \texttt{gcd}(155, \texttt{rem}(248, 155) = 93) \\
&= \texttt{gcd}(93, \texttt{rem}(155, 93) = 62) \\
&= \texttt{gcd}(62, \texttt{rem}(93, 62) = 31) \\
&= \texttt{gcd}(31, \texttt{rem}(62, 31) = 0) \\
&= 31
\end{aligned}$$

As another example, applying Euclid's algorithm to 26 and 21 gives

$$\texttt{gcd}(26, 21) = \texttt{gcd}(21, 5) = \texttt{gcd}(5, 1) = 1,$$

which means that 26 and 21 share no factors other than 1. When a pair of numbers has this property, we say they are *relatively prime* or *coprime*.

5.2.2 The Extended Euclidean Algorithm

We will get a lot of mileage out of the following key fact:

Theorem 5.2.2 (Bezout's Lemma)**.** *For any two integers a and b,*

$$\texttt{gcd}(a, b) = sa + tb,$$

for some integers s and t.

We already know from Lemma 5.1.2 that, for any integers s and t, the combination $sa+tb$ must be divisible by any common factor of a and b, so it is certainly divisible by the greatest of these common divisors. Multiplying both sides by a constant gives:

Corollary 5.2.3. *Let a, b, and n be integers. Then there exist integers s and t such that $n = sa+tb$ if and only if n is a multiple of $\texttt{gcd}(a, b)$.*

We'll prove Theorem 5.2.2 directly by explaining how to find s and t. This job is tackled by a mathematical tool that dates back to sixth-century India, where it was called *kuttaka*, which means "the Pulverizer." Today, the Pulverizer is more commonly known as the "Extended Euclidean GCD Algorithm," because it is so close to Euclid's algorithm.

For example, following Euclid's algorithm, we can compute the gcd of 259 and 70 as follows:

$$\begin{aligned}
\texttt{gcd}(259, 70) &= \texttt{gcd}(70, 49) && \text{since } \texttt{rem}(259, 70) = 49 \\
&= \texttt{gcd}(49, 21) && \text{since } \texttt{rem}(70, 49) = 21 \\
&= \texttt{gcd}(21, 7) && \text{since } \texttt{rem}(49, 21) = 7 \\
&= \texttt{gcd}(7, 0) && \text{since } \texttt{rem}(21, 7) = 0 \\
&= 7
\end{aligned}$$

The Extended Euclidean Algorithm goes through the same steps, but requires some extra bookkeeping along the way: as we compute $\texttt{gcd}(a, b)$, we keep track of how to write each of the remainders (49, 21, and 7, in the example) in the form $sa + tb$. This is worthwhile, because our objective is to write the last nonzero remainder, which is the gcd, in such a way. For our example, here is this extra bookkeeping:

x	y	$\texttt{rem}(x, y)$	$=$	$x - q \cdot y$
259	70	49	$=$	$a - 3 \cdot b$
70	49	21	$=$	$b - 1 \cdot 49$
			$=$	$b - 1 \cdot (a - 3 \cdot b)$
			$=$	$-1 \cdot a + 4 \cdot b$
49	21	7	$=$	$49 - 2 \cdot 21$
			$=$	$(a - 3 \cdot b) - 2 \cdot (-1 \cdot a + 4 \cdot b)$
			$=$	$\boxed{3 \cdot a - 11 \cdot b}$
21	7	0		

We began by initializing two variables, $x = a$ and $y = b$. In the first two columns above, we carried out Euclid's algorithm. At each step, we computed $\texttt{rem}(x, y)$ which equals $x - \texttt{qnt}(x, y) \cdot y$. Then, we replaced x and y by the formula in terms of a and b, which we already had computed. After simplifying, we were left with a formula in terms of a and b that gives us $\texttt{rem}(x, y)$, as desired. The final solution is boxed.

This should make it pretty clear how and why the Extended Euclidean Algorithm works. Since the Extended Euclidean Algorithm requires only a little more computation than Euclid's algorithm,

you can "pulverize" very large numbers very quickly by using this algorithm. As we will soon see, its speed makes the Extended Euclidean Algorithm a very useful tool in the field of cryptography.

5.2.3 Properties of the Greatest Common Divisor

It can help to have some basic gcd facts on hand:

Lemma 5.2.4.

a) $\gcd(ka, kb) = k \cdot \gcd(a, b)$ *for all* $k > 0$.

b) $(d \mid a \wedge d \mid b) \leftrightarrow d \mid \gcd(a, b)$.

c) If $\gcd(a, b) = 1$ *and* $\gcd(a, c) = 1$*, then* $\gcd(a, bc) = 1$.

d) If $a \mid bc$ *and* $\gcd(a, b) = 1$*, then* $a \mid c$.

Showing how all these facts follow from Theorem 5.2.2 is a good exercise.

These properties are also simple consequences of the fact that integers factor into primes in a unique way. We'll need some of these facts to prove unique factorization, so proving them by appeal to unique factorization would be circular.

5.3 The Fundamental Theorem of Arithmetic

Compare with Section 9.4 from Mathematics for Computer Science

There is an important fact about primes that you probably already know: every positive integer number has a unique prime factorization. So every positive integer can be built up from primes in exactly one way. These quirky prime numbers are the building blocks for the integers.

Since the value of a product of numbers is the same if the numbers appear in a different order, there usually isn't a unique way to express a number as a product of primes. For example, there are three ways to write 12 as a product of primes:

$$12 = 2 \cdot 2 \cdot 3 = 2 \cdot 3 \cdot 2 = 3 \cdot 2 \cdot 2.$$

What's unique about the prime factorization of 12 is that any product of primes equal to 12 will have exactly one 3 and two 2's. This means that if we sort the primes by size, then the product really will be unique.

Let's state this more carefully. A sequence of numbers is *weakly increasing* when each number in the sequence is at less than or equal to the numbers after it. Note that a sequence of just one number as well as a sequence of no numbers—the empty sequence—is weakly increasing by this definition.

Theorem 5.3.1 (The Fundamental Theorem of Arithmetic)**.** *Every positive integer can be written uniquely as a product of a weakly increasing sequence of primes.*

For example, 75237393 is the product of the weakly increasing sequence of primes

$$3, 7, 7, 7, 11, 17, 17, 23,$$

and no other weakly increasing sequence of primes will give 75237393.

Notice that the theorem would be false if 1 were considered a prime; for example, 15 could be written as $3 \cdot 5$, or $1 \cdot 3 \cdot 5$, or $1 \cdot 1 \cdot 3 \cdot 5$, ...

There is a certain wonder in unique factorization. It's a mistake to take it for granted, even if you've known it since you were in a crib. In fact, unique factorization actually fails for many integer-like sets of numbers, such as the complex numbers of the form $n + m\sqrt{-5}$ for $m, n \in \mathbb{Z}$.

The Fundamental Theorem of Arithmetic is also called the *Unique Factorization Theorem*, which is a more descriptive and less pretentious name—but we really want to get your attention to the importance and non-obviousness of unique factorization.

5.3.1 Proving Unique Factorization

The Fundamental Theorem is not hard to prove, but we'll need a couple of preliminary facts.

Lemma 5.3.2. *If p is a prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.*

Lemma 5.3.2 follows immediately from Unique Factorization: the primes in the product ab are exactly the primes from a and from b. But proving the lemma this way would be cheating: we're going to need this lemma to prove Unique Factorization, so it would be circular to assume it. Instead, we'll use the properties of gcd's to give an easy, noncircular way to prove Lemma 5.3.2.

Proof. One case is if $\texttt{gcd}(a, p) = p$. Then the claim holds because a is a multiple of p.

Otherwise, $\texttt{gcd}(a, p) \neq p$. In this case $\texttt{gcd}(a, p)$ must be 1, since 1 and p are the only positive divisors of p. Now $1 = \texttt{gcd}(a, p)$ can be written as $1 = sa + tp$ for some integers s and t. Then $b = s(ab) + (tb)p$. Since p divides both ab and p, it also must divide b. □

A routine induction argument extends this statement to:

Lemma 5.3.3. *Let p be a prime. If $p \mid a_1 a_2 \cdots a_n$, then p divides a_i for some index i with $1 \leq i \leq n$.*

Now we're ready to prove the Fundamental Theorem of Arithmetic.

Proof. First, recall that every positive integer can be expressed as a product of primes. So we just have to prove this expression is unique.

The proof is by contradiction: assume, contrary to the claim, that there exist positive integers that can be written as products of primes in more than one way. Then there is a smallest positive integer with this property. Call this integer n, and let

$$\begin{aligned} n &= p_1 \cdot p_2 \cdots p_j, \\ &= q_1 \cdot q_2 \cdots q_k, \end{aligned}$$

where both products are in weakly increasing order and $p_j \leq q_k$.[1] If $q_k = p_j$, then n/q_k would also be the product of different weakly increasing sequences of primes, namely,

$$p_1 \cdots p_{j-1},$$

$$q_1 \cdots q_{k-1}$$

Noting that $n/q_k < n$, this can't be true since n was assumed to be the smallest positive integer with this property. Hence, we conclude that $p_j < q_k$. Since the p_i's are weakly increasing, all the p_i's are less than q_k. But

$$q_k \mid n = p_1 \cdot p_2 \cdots p_j,$$

so Lemma 5.3.3 implies that q_k divides one of the p_i's, which contradicts the fact that q_k is greater than all of them. □

5.4 Modular Arithmetic

5.4.1 Modular Congruence

Compare with Section 9.6 from Mathematics for Computer Science

On the first page of his masterpiece on number theory, *Disquisitiones Arithmeticae*, Gauss introduced the notion of *congruence*. Gauss said that a is *congruent to* b *modulo* n whenever $n \mid (a - b)$. n is called the *modulus* (plural: *moduli*) of the congruence, and such a congruence is written

$$a \equiv b \pmod{n}.$$

For example:

$$29 \equiv 15 \pmod{7} \qquad \text{because} \qquad 7 \mid (29 - 15) = 14.$$

It's not useful to allow a modulus $n \leq 0$, so we will assume from now on that moduli are always positive.

There is a close connection between congruences and remainders:

Lemma 5.4.1 (The Remainder Lemma)**.**

$$a \equiv b \pmod{n} \leftrightarrow \mathit{rem}(a, n) = \mathit{rem}(b, n).$$

Proof. By the Division Algorithm, there exist unique pairs of integers q_1, r_1 and q_2, r_2 such that:

$$a = q_1 n + r_1,$$

$$b = q_2 n + r_2,$$

[1] If $q_k \leq p_j$, just switch which sequence is labeled with p and which sequence is labeled with q. This is a common trick when working with arbitrary labels

where $0 \leq r_1, r_2 < n$. Subtracting the second equation from the first gives:

$$a - b = (q_1 - q_2)n + (r_1 - r_2),$$

where $-n < (r_1 - r_2) < n$. Now $a \equiv b \pmod{n}$ if and only if n divides the left-hand side of this equation. This is true if and only if n divides the right-hand side, which holds if and only if $r1 - r2$ is a multiple of n. But the only multiple of n strictly between $-n$ and n is 0, so $r1 - r2$ must in fact equal 0, that is, $r_1 = r_2$. But $r_1 = \texttt{rem}(a, n)$ and $r_2 = \texttt{rem}(b, n)$ by construction, so this happens exactly when $\texttt{rem}(a, n) = \texttt{rem}(b, n)$. □

Using this, we can see that

$$29 \equiv 15 \pmod{7} \qquad \text{because} \qquad \texttt{rem}(29, 7) = 1 = \texttt{rem}(15, 7).$$

Notice that even though "(mod 7)" appears on the end, the $\equiv$ symbol isn't any more strongly associated with the 15 than with the 29. It would probably be clearer to write $29 \equiv_{\texttt{mod } 7} 15$, for example, but the notation with the modulus at the end is firmly entrenched, and we'll just live with it.

We'll make frequent use of an immediate corollary of The Remainder Lemma:

Corollary 5.4.2.

$$a \equiv \texttt{rem}(a, n) \pmod{n}$$

The Remainder Lemma explains why the congruence relation has properties like an equality relation. In particular, the following properties follow immediately:

Lemma 5.4.3.

$$\begin{aligned} a &\equiv a \pmod{n} && \text{(reflexivity)} \\ a \equiv b &\leftrightarrow b \equiv a \pmod{n} && \text{(symmetry)} \\ ((a \equiv b) \wedge (b \equiv c)) &\rightarrow (a \equiv c) \pmod{n} && \text{(transitivity)} \end{aligned}$$

Because of this, congruence mod n for a *fixed n value* is an equivalence relation on the integers, so it defines a partition of the integers into n sets so that congruent numbers are all in the same set. For example, suppose that we're working modulo 3. Then we can partition the integers into 3 sets as follows:

$$\begin{aligned} &\{ \ \ldots, \ -6, \ -3, \ 0, \ 3, \ 6, \ 9, \ \ldots \ \} \\ &\{ \ \ldots, \ -5, \ -2, \ 1, \ 4, \ 7, \ 10, \ \ldots \ \} \\ &\{ \ \ldots, \ -4, \ -1, \ 2, \ 5, \ 8, \ 11, \ \ldots \ \} \end{aligned}$$

according to whether their remainders on division by 3 are 0, 1, or 2. The upshot is that when arithmetic is done modulo n, there are really only n different kinds of numbers to worry about, because there are only n possible remainders. In this sense, modular arithmetic is a simplification of ordinary arithmetic.

The next most useful fact about congruences is that they are *preserved* by addition and multiplication:

Lemma 5.4.4 (The Congruence Lemma)**.** *If* $a \equiv b \pmod{n}$ *and* $c \equiv d \pmod{n}$*, then*

$$a + c \equiv b + d \pmod{n}, \tag{5.2}$$

$$ac \equiv bd \pmod{n}. \tag{5.3}$$

Proof. Let's start with (5.2). Since $a \equiv b \pmod{n}$, we have by definition that $n \mid (b - a) = (b + c) - (a + c)$, so

$$a + c \equiv b + c \pmod{n}.$$

Since $c \equiv d \pmod{n}$, the same reasoning leads to

$$b + c \equiv b + d \pmod{n}.$$

Now transitivity (from Lemma 5.4.3) gives

$$a + c \equiv b + d \pmod{n}.$$

The proof for (5.3) is virtually identical, using the fact that if n divides $(b-a)$, then it certainly also divides $(b - a)c = (bc - ac)$. □

5.4.2 Arithmetic Operations Modulo n

Compare with Section 9.7 from Mathematics for Computer Science

The Congruence Lemma says that two numbers are congruent if and only if their remainders are equal, so we can understand congruences by working out arithmetic with remainders. And if all we want is the remainder modulo n of a series of additions, multiplications, subtractions applied to some numbers, we can take remainders at every step so that the entire computation only involves number in the range from 0 to $n - 1$.

General Principle of Remainder Arithmetic

To find the remainder on division by n of the result of a series of additions and multiplications, applied to some integers

- replace each integer operand by its remainder modulo n,
- keep each result of an addition or multiplication in the range from 0 to $n - 1$ by immediately replacing any result outside that range by its remainder modulo n.

For example, suppose we want to find

$$\texttt{rem}((44427^{3456789} + 15555858^{5555})403^{6666666}, 36). \tag{5.4}$$

This looks really daunting if you think about computing these large powers and then taking remainders. For example, the decimal representation of $44427^{3456789}$ has about 20 million digits, so we certainly don't want to go that route. But remembering that integer exponents specify a series of multiplications, we follow the General Principle and replace the numbers being multiplied by their remainders. Since

$$\begin{aligned} \texttt{rem}(44427, 36) &= 3, \\ \texttt{rem}(15555858, 36) &= 6, \text{ and} \\ \texttt{rem}(403, 36) &= 7, \end{aligned}$$

we find that (5.4) equals the remainder modulo 36 of

$$(3^{3456789} + 6^{5555})7^{6666666}. \tag{5.5}$$

That's a little better, but $3^{3456789}$ has about a million digits in its decimal representation, so we still don't want to compute that. But let's look at the remainders of the first few powers of 3:

$$\begin{aligned} \texttt{rem}(3^1, 36) &= 3 \\ \texttt{rem}(3^2, 36) &= 9 \\ \texttt{rem}(3^3, 36) &= 27 \\ \texttt{rem}(3^4, 36) &= 9. \end{aligned}$$

We got a repeat of the second step, $\texttt{rem}(3^2, 36)$ after just two more steps. This means means that starting at 3^2, the sequence of remainders of successive powers of 3 will keep repeating every 2 steps. So a product of an odd number of at least three 3's will have the same remainder on division by 36 as a product of just three 3's. Therefore,

$$\texttt{rem}(3^{3456789}, 36) = \texttt{rem}(3^3, 36) = 27.$$

What a win!

Powers of 6 are even easier because $\texttt{rem}(6^2, 36) = 0$, so 0's keep repeating after the second step. Powers of 7 repeat after six steps, but on the fifth step you get a 1, that is $\texttt{rem}(7^6, 36) = 1$, so (5.5) successively simplifies to be the following terms:

$$\begin{aligned} & (3^{3456789} + 6^{5555})7^{6666666} \\ &= (3^{3456789} + 6^2 \cdot 6^{5553})(7^6)^{1111111} \\ &\equiv (3^3 + 0 \cdot 6^{5553})1^{1111111} \pmod{36} \\ &= 27 \end{aligned}$$

Notice that it would be a *disastrous blunder* to replace an exponent by its remainder. The general principle applies to numbers that are operands of plus and times, whereas the exponent is a number that controls how many multiplications to perform. **Watch out for this**.

The ring $\mathbb{Z}_n$

It's time to be more precise about the general principle and why it works. To begin, let's introduce the notation $+_n$ for doing an addition and then immediately taking a remainder modulo n, as specified by the general principle; likewise for multiplying:

$$i +_n j = \mathtt{rem}(i + j, n),$$
$$i \cdot_n j = \mathtt{rem}(i \cdot j, n).$$

Now the General Principle is simply the repeated application of the following lemma.

Lemma 5.4.5.

$$\mathtt{rem}(i + j, n) = \mathtt{rem}(i, n) +_n \mathtt{rem}(j, n) \tag{5.6}$$
$$\mathtt{rem}(i \cdot j, n) = \mathtt{rem}(i, n) \cdot_n \mathtt{rem}(j, n) \tag{5.7}$$

Proof. By Corollary 5.4.2, $i \equiv \mathtt{rem}(i, n)$ and $j \equiv \mathtt{rem}(j, n)$, so by The Congruence Lemma,

$$i + j \equiv \mathtt{rem}(i, n) + \mathtt{rem}(j, n) \pmod{n}.$$

By Corollary 5.4.2 again, the remainders on each side of this congruence are equal, which immediately gives (5.6). An identical proof applies to (5.7). □

The set of integers in the range from 0 to $n - 1$ together with the operations $+_n$ and $\cdot_n$ is referred to as $\mathbb{Z}_n$, the ring of integers modulo n. As a consequence of Lemma 5.4.5, the familiar rules of arithmetic hold in $\mathbb{Z}_n$, for example:

$$(i \cdot_n j) \cdot_n k = i \cdot_n (j \cdot_n k).$$

These subscript-n's on arithmetic operations really clog things up, so instead we'll just write "$(\mathbb{Z}_n)$" on the side to get a simpler looking equation:

$$(i \cdot j) \cdot k = i \cdot (j \cdot k) \quad (\mathbb{Z}_n).$$

In particular, all of the following equalities are true in $\mathbb{Z}_n$:

$$\begin{aligned}
(i \cdot j) \cdot k &= i \cdot (j \cdot k) && \text{(associativity of } \cdot\text{)},\\
(i + j) + k &= i + (j + k) && \text{(associativity of } +\text{)},\\
1 \cdot k &= k && \text{(identity for } \cdot\text{)},\\
0 + k &= k && \text{(identity for } +\text{)},\\
k + (-k) &= 0 && \text{(inverse for } +\text{)},\\
i + j &= j + i && \text{(commutativity of } +\text{)},\\
i \cdot (j + k) &= (i \cdot j) + (i \cdot k) && \text{(distributivity)},\\
i \cdot j &= j \cdot i && \text{(commutativity of } \cdot\text{)}.
\end{aligned}$$

Associativity implies the familiar fact that it's safe to omit the parentheses in products:

$$k_1 \cdot k_2 \cdot \ldots \cdot k_m$$

comes out the same in $\mathbb{Z}_n$ no matter how it is parenthesized.

The overall theme is that remainder arithmetic is a lot like ordinary arithmetic, but is a powerful tool that can help simplify complex computations.

Chapter 6

Combinatorics

Chapter 7

Measuring Asymptotic Growth

7.1 Big-Oh Notation

Source: This section adapts ideas and examples from cletus, "What is a plain English explanation of 'Big O' notation?," *Stack Overflow*, Jan. 28, 2009, edited May 29, 2022, https://stackoverflow.com/questions/487258/what-is-a-plain-english-explanation-of-big-o-notation. Stack Overflow content is licensed under CC BY-SA 4.0.

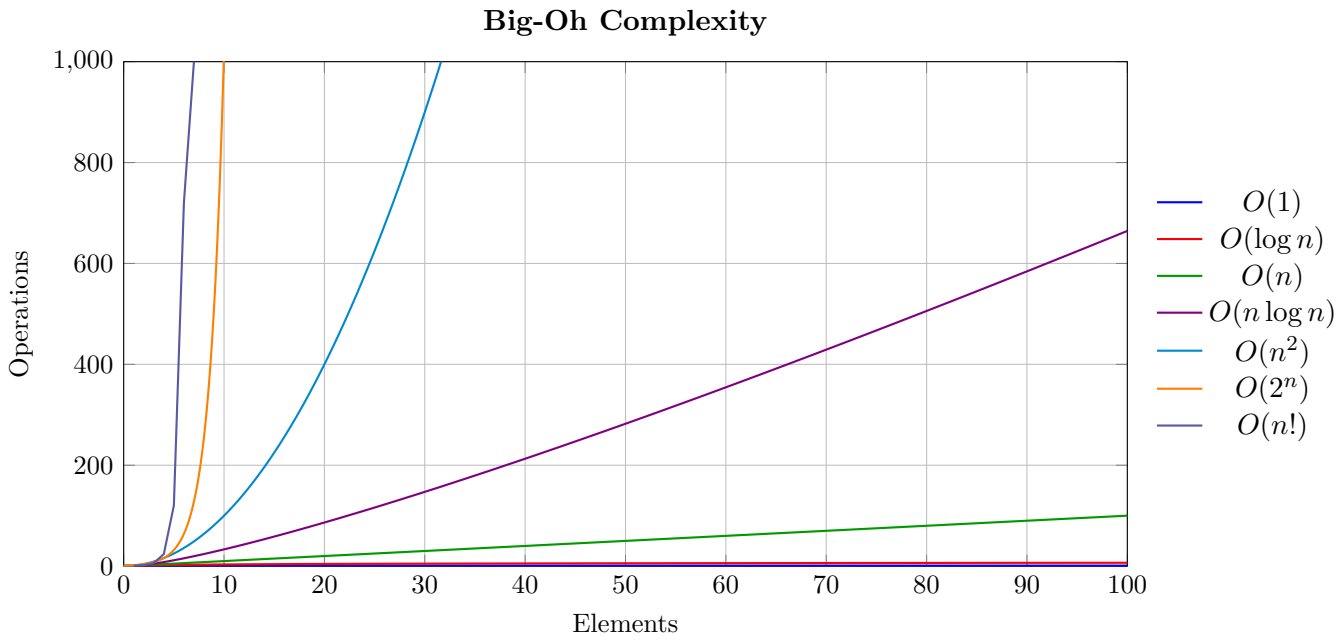


Figure 7.1: Common Big-Oh growth rates. Graph recreated from the visual included in cletus, "What is a plain English explanation of 'Big O' notation?," *Stack Overflow*, https://stackoverflow.com/questions/487258/what-is-a-plain-english-explanation-of-big-o-notation.

One big goal in mathematics for computer science is learning how to compare different ways of doing a job. If two students each finish a task in ten minutes, that may sound like a tie. But if

one student organized ten papers and the other organized ten thousand, those ten minutes do not mean the same thing. Big-Oh notation gives us a way to talk about how the amount of work grows as the input gets bigger.

At its core, Big-Oh answers a simple question: if the problem gets much bigger, how much more work will it take? That is why Big-Oh is so useful in math and computer science. We usually do not care about the exact number of seconds on one laptop. We want to know whether doubling the size of the job roughly doubles the work, squares the work, or makes the work grow very fast.

There are three main ideas here. First, Big-Oh is *relative*: it compares one kind of growth to another, so the comparison only makes sense when we are measuring similar things. Second, Big-Oh is a *representation*: we choose what counts as "work" and use that as our measuring stick. For one problem we might count arithmetic steps. For another we might count comparisons. For another we might count memory accesses. Third, Big-Oh is about *complexity*: it tells us how the amount of work changes when the input size changes.

An everyday example comes from ordinary arithmetic. Suppose you add two six-digit numbers such as 123456 and 789012. Written the usual way, the calculation looks like this:

$$\begin{array}{r} 123456 \\ +\,789012 \\ \hline 912468 \end{array}$$

You move column by column from right to left, adding $6 + 2$, then $5 + 1$, then $4 + 0$, and so on. So the amount of work is tied directly to the number of digits. If the numbers have n digits, then addition takes work proportional to n. So addition has linear growth, which we write as $O(n)$. Subtraction behaves the same way.

Multiplication is different. For example, if we multiply 123 by 456, we can write:

$$\begin{array}{r} 123 \\ \times\,456 \\ \hline 738 \\ 615 \\ 492 \\ \hline 56088 \end{array}$$

Using the grade-school method, each digit in one number is multiplied by each digit in the other number. The 6 in 456 multiplies all three digits of 123, then the 5 does the same, then the 4 does the same. If both numbers have n digits, that leads to about n^2 digit-by-digit multiplications, plus some extra additions to combine the partial products. Those extra steps matter if you want an exact count, but they do not change the main growth pattern. So grade-school multiplication is $O(n^2)$.

This leads to an important idea: in Big-Oh, we focus on the part of the expression that matters most when the input becomes large. For example, if an algorithm takes $n^2 + 2n$ steps, the n^2 term

will matter more than the $2n$ term once n gets large. When n is very large, the extra $2n$ is only a small part of the total. That is why both $n^2 + 2n$ and $3n^2 + 100n + 7$ have Big-Oh $O(n^2)$.

This is also why adding a small extra amount does not change the big picture. The functions x^2 and $x^2 + 1$ are not exactly the same, but the $+1$ matters less and less as x grows. In the same way, changing a constant multiplier does not change the Big-Oh class. For large-scale growth, $100x^2$ and x^2 behave the same way.

In this text, we use Big-Oh in its standard math meaning: an eventual upper bound up to a constant multiple. Informally, saying $f = O(g)$ means that once the input is large enough, the values of f do not grow past a constant multiple of the values of g.

Definition 7.1.1 (Big-Oh)**.** Given two functions f and g, we say that f is *big-oh* of g, written $f = O(g)$ or $f(x) = O(g(x))$, if there exist a real number k and a positive real number C so that, whenever $x \geq k$, we also get that

$$|f(x)| \leq C|g(x)|.$$

Put formally into logic, this definition becomes

$$\exists\, k \in \mathbb{R}, \exists\, C \in \mathbb{R}^+, \forall\, x \in \mathbb{R}\ [\,(\,x \geq k\,) \rightarrow (\,|f(x)| \leq C|g(x)|\,)\,] \tag{7.1}$$

For example, $x^2 + 1 = O(x^2)$. To show this, we need to find a constant C and a starting point k so that

$$|x^2 + 1| \leq C|x^2|$$

whenever $x \geq k$. If we choose $C = 2$ and $k = 1$, then for every $x \geq 1$ we get

$$|x^2 + 1| \leq 2|x^2|.$$

This means that after x reaches 1, the function $x^2 + 1$ never grows bigger than twice x^2. So $x^2 + 1$ is $O(x^2)$.

The other direction is also true: $x^2 = O(x^2 + 1)$. That does not mean the two functions are equal. It means each one is eventually no bigger than a constant multiple of the other. This is why Big-Oh is about comparing growth, not about saying two functions are exactly the same.

For polynomials, the term with the largest power is the one that matters most when x gets large. For example, in $3x^2 + 100x + 7$, the x^2 term will eventually matter more than the x term and the constant term. That is why the degree of a polynomial tells you its Big-Oh class. A constant function is $O(1)$, a linear function is $O(x)$, a quadratic function is $O(x^2)$, and so on. To see a proof that the leading term of a polynomial always dominates, see Appendix A.1.

7.2 Computing the Big-Oh of Compound Functions

The direct proof of Theorem A.1.3 is fairly complicated and uses the Generalized Triangle Inequality. In practice, we usually break a compound function into smaller functions, find the big-oh of

each one, and then combine those results.

The table below lists several common functions used to measure against for big-oh. They are arranged from larger big-oh to smaller big-oh. We say that f has a "smaller" big-oh than g, or $O(f) < O(g)$, if $f = O(g)$ but $g \neq O(f)$. For example, $x^2 = O(x^3)$ by picking $C = 1$ and $k = 0$, but no choice of C or k will give you the reverse, so $x^3 \neq O(x^2)$. The second column of the table states how to compare different functions of the same form.

Basic Function	How to Compare Different Functions of this Form
x^x	Not applicable
$x!$	Not applicable
a^x for $a > 0$	If $a < b$ then $O(a^x) < O(b^x)$
x^n for $n > 0$	If $m < n$ then $O(x^m) < O(x^n)$
$\log_a(x)$ for $a > 0$	All logarithms have the same big-oh, written $O(\log(x))$
$O(c)$ for $c \in \mathbb{R}$	All constant polynomials are $O(1)$

Table 7.1: Common functions arranged by decreasing big-oh value

With these basic functions in hand, we can use the following rules to compute the big-oh of more complicated functions:

1. If $f = O(a)$ and $g = O(b)$, then $f + g = \texttt{max}(O(a), O(b))$
2. If $f = O(a)$ and $g = O(b)$, then $fg = O(ab)$
3. If $f = O(a)$ and $g = O(b)$, then $f \circ g = O(a \circ b)$

These properties make it much easier to find the big-oh of a polynomial. Property 1 tells us that the big-oh of a polynomial is the big-oh of its leading term, and Property 2 tells us that the constant in front of a term does not change that term's big-oh.

In practice, these properties are how mathematicians actually compute the big-oh of compound functions. Once we know the big-oh of the smaller pieces, we can find the big-oh of the whole function.

7.3 Computational Complexity of Algorithms

In computer science, the same basic idea is used to compare algorithms. The exact running time of a program depends on the machine, the programming language, the compiler, the current load on the computer, and many other details. So instead of measuring raw seconds, we usually count the number of *operations* an algorithm performs as the input grows. This measurement is called the *computational complexity* or *time complexity* of the algorithm.[1]

[1] There is also a concept of the "space complexity" of an algorithm: the amount of memory required to run the algorithm. Generally speaking, time and space can be traded against each other; an algorithm can sometimes be made faster by storing more information, or made smaller by recomputing information instead of storing it.

What counts as an operation depends on the problem. Sometimes the most expensive step is arithmetic. Sometimes it is comparing two objects. Sometimes it is moving data in memory. This is one reason Big-Oh is not an exact stopwatch reading. Different reasonable ways of counting may give slightly different formulas, but they usually lead to the same Big-Oh class.

When people talk about an algorithm's Big-Oh, they also often care about which case is being measured. The *best case* is what happens when everything goes as well as possible. The *worst case* is the most work the algorithm might need for an input of a given size. The *expected case* is the amount of work we would usually expect. In many applications, the worst case matters most because it tells us what can happen when things go badly.

One of the most familiar examples is searching through your phone contacts. Suppose your contacts are sorted alphabetically and you want to find "John Smith." You would not start at the top and read every name one by one. A better idea is to jump near the middle, decide whether "Smith" should come before or after the name you see, then ignore half of the remaining list and repeat. This is called *binary search*.

Binary search is powerful because each comparison cuts the remaining search space roughly in half. If the contact list has 3 names, you need at most 2 comparisons. If it has 7 names, you need at most 3. If it has 15 names, you need at most 4. For a list with about 1,000,000 names, it takes only about 20 comparisons. That growth pattern is logarithmic, written $O(\log n)$.

Notice how dramatic this is. An input that is much larger does not require that much more work. It only requires a few more comparisons. That is why logarithmic-time algorithms are usually considered very efficient.

Now imagine the reverse problem. Suppose you have a phone number and want to find the matching person in your contacts. Your contacts are usually sorted by name, not by phone number, so halving the search space no longer helps. You may need to start at the beginning and check one contact after another until you find a match. That means the work grows in step with the number of entries, so the search is $O(n)$. In the best case the answer is the first entry, which is $O(1)$, but in the typical and worst cases you should expect linear growth.

At the other extreme are problems whose growth becomes enormous very quickly. One famous example is the *traveling salesman problem*: given a list of towns and the distances between them, find the shortest route that visits every town. Even for a fairly small number of towns, the number of possible tours grows very fast. With 5 towns, the number of possibilities is on the order of $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1$. With 6 towns it is on the order of 6!, with 7 towns on the order of 7!, and so on. This kind of growth is called factorial growth and is written $O(n!)$. It becomes unmanageable very quickly.

These examples are why the standard growth families matter so much: constant time $O(1)$, logarithmic time $O(\log n)$, linear time $O(n)$, quadratic time $O(n^2)$, exponential time, and factorial time $O(n!)$ do not merely differ by notation. They describe genuinely different experiences as the input grows.

Algorithms whose running time is bounded by $O(n^a)$ for some exponent a are said to run in

polynomial time. Examples include $O(n)$ and $O(n^2)$. Many problems that can be solved in polynomial time are considered practical, even when the input becomes fairly large. By contrast, problems with exponential or factorial growth often become impractical much sooner. This difference is one reason modern cryptography works at all: some tasks are easy to do, but undoing them is believed to be hard.

With that intuition in hand, we can now compute Big-Oh for specific algorithms. For example, let `sum` be the following algorithm that takes as input a list of length n of integers and returns the sum of those integers.

For example, let `sum` be the following algorithm that takes as input a list of length n of integers and returns the sum of those integers.

`sum(list):`

1. Set the variable `result` to be equal to the first element of `list`
2. For each remaining element:
   (a) take the sum of the current element with `result`
   (b) update `result` to the new sum
3. return `result`

This algorithm uses one operation for the first element of the list, two for each of the remaining $n-1$ elements, and an additional operation to return the result. This gives us a total of $1+2(n-1)+1 = 1+2n-2+1 = 2n$ operations. Since $2n = O(n)$, we would say that this algorithm requires $O(n)$ operations, or more simply, that the algorithm is $O(n)$.

One could argue that accessing each element from the list is an operation, so the total number of operations should be $2+3(n-1)+1 = 3n$. However $2n$ and $3n$ are both $O(n)$, so our computational complexity would be the same in either case.

7.4 Computing the Big-Oh of Compound Algorithms

It is not uncommon to build an algorithm out of other algorithms. For example, the following one-line algorithm keeps the variable `num` bounded between 1 and 10:

1. Set `num` to be the value `min`(10, `max`(1, `num`))

How do we compute the big-oh of such an algorithm? The first and simplest rule is as follows:

> **Rule 1:** If algorithm A is $O(a)$ and algorithm B is $O(b)$, then running algorithm A then algorithm B is $O(\texttt{max}(a, b))$.

In the example above, we are running `max` first, then plugging the result into `min`, then finally assigning that result to the variable `num`. So for the example above, assuming the assignment operator is $O(1)$, the `min` algorithm is $O(\log(n))$, and the `max` algorithm is $O(n^2)$ (in practice both `min` and `max` should be $O(1)$ for comparing two numbers, but we use other big-oh values here to illustrate the point), the whole algorithm would be the maximum of $O(1)$, $O(\log(n))$, and $O(n^2)$. Comparing these functions using Table 7.1 gives a total big-oh of $O(n^2)$.

Things get a bit more complicated when you start introducing *loops*. In computer science, a loop is when a portion of code is repeated. The number of times the code is repeated varies based on the type of loop used, the conditions given to the loop, and the existence of lines of code that break out of the loop prematurely. There are three main types of loops that we will discuss here. Their names may vary based on programming language, but generally they are known as *while loops*, *for loops*, and *recursive loops*.

While Loop: A *while loop* is a loop that executes indefinitely until certain conditions given by the programmer are met. There is no general rule for computing the big-oh of a while loop since the number of iterations depends heavily on exactly what the exit conditions are and how they interact with the code inside the loop.

For Loop: A *for loop* is a loop that falls into one of two categories:

a) A loop that has a `start`, `stop`, and `increment` value. This kind of for loop has an index variable that is set to `start`, increases by `increment` for each iteration of running the block of code, then exits once the index variable meets or exceeds `stop`. If this kind of for loop is set up as intended, it should run for exactly $\left|\frac{\texttt{start}-\texttt{stop}}{\texttt{increment}}\right|$ iterations, rounded up (or down depending on the programming language) if not an integer. This value may change if the index variable is manipulated within the code contained in the loop. This kind of for loop is more common with procedural programming languages such as C.

b) A loop that has an iterable of object and runs once for each object in the iterable. This kind of loop will run for exactly the same number of iterations as the number of objects in the iterable. This kind of for loop is more common with object-oriented programming languages such as Python.

Recursive Loop: A *recursive loop* works similarly to mathematical induction where the result of running a block of code depends on running that same block of code with different inputs. Generally speaking, a recursive loop will have one or more base cases where the programmer manually tells the program what to return and one or more recursive cases where the program will call itself with different inputs. As with while loops, the big-oh of a recursive loop can vary wildly depending on the way the loop is set up.

Without any additional information and assuming there are no lines of code which break out of the loop prematurely, for loops are the only kind of loop that are easy to predict the number of iterations that will run. That being said, the following rule works for any loop:

> **Rule 2:** If a loop runs for n iterations, and each iteration is $O(a)$, then the whole loop is $O(na)$.

For example, consider the following algorithm that takes in a positive integer n and returns its factorial, $n!$:

`factorial`(n):

1. Set `result` to 1
2. For `index` with `start` $= n$, `stop` $= 0$, `increment` $= -1$:
   (a) Multiply `result` with `index`
   (b) Set `result` to the new product
3. Return `result`

`factorial` contains one assignment operation, one for loop, and one return operation. The for loop runs for $\left|\frac{n-0}{-1}\right| = n$ iterations, and each iteration contains two operations (not counting updating the index variable and checking it against the stop condition which together are always $O(1)$ anyway). Hence, by Rule 2, the loop is $O(2n) = O(n)$. By Rule 1, the whole algorithm's big-oh is the maximum of $O(1)$, $O(n)$, and $O(1)$. Comparing these functions using Table 7.1 gives a total big-oh of $O(n)$, where n is the input number.

It is important to note the importance of stating, "where n is the input number," at the end of stating the big-oh. Sometimes in computer science, the big-oh will depend on the *value* of the input and sometimes it will depend on the *number* of inputs. For example, consider the following algorithm that makes use of `factorial`:

`many_factorials(input_list)`:

1. Set `result` to be an empty list
2. For each `element` of `input_list`:
   (a) Compute `factorial(element)`
   (b) Append the result to `result`
3. Return `result`

`factorial` contains one assignment operation, one for loop, and one return operation. The for loop runs for m iterations, where m is the number of elements in `input_list` and each iteration contains one use of `factorial` and one other operation. By Rule 1, each iteration of the loop has a big-oh equal to the maximum of:

- $O(n)$ (from `factorial`) where n is the `element` from `input_list` being computed
- $O(1)$ (from the append operation)

Comparing these functions using Table 7.1 gives a total big-oh of $O(n)$, where n is the input number. Hence, by Rule 2, the loop is $O(mn)$. It should be noted that n changes with each iteration of the loop, but since big-oh is an upper-bound, we can just let n represent the largest `element` in `input_list`. By Rule 1, the whole algorithm's big-oh is the maximum of $O(1)$, $O(mn)$, and $O(1)$. This gives a total big-oh of $O(mn)$, where m is the number of `elements` in `input_list` and n is the largest element in `input_list`.

Appendix A

Supplementary Material

A.1 Proving Polynomial Big-Oh Depends Only on the Leading Term

We first need the following lemma.

Lemma A.1.1 (Triangle Inequality)**.** *For any real numbers a and b,*

$$|a + b| \leq |a| + |b|.$$

Proof. We proceed by cases.

Case 1. Suppose that a and b are both positive. Then $a + b$ is positive, so

$$|a + b| = a + b = |a| + |b|.$$

Case 2. Suppose that exactly one of a or b is positive and one is negative. Without loss of generality, we will assume that b is negative. We have two subcases

Case 2.1. Assume that $|a| > |b|$, that is $a+b > 0$. Then, since a is positive and b is negative, we have that $a = |a|$ and $b < |b|$. This gives us that

$$|a + b| = a + b < |a| + |b|.$$

Case 2.2. Assume that $|a| < |b|$, that is $a+b < 0$. Then, since a is positive and b is negative, we get that $-a < |a|$ and $-b = |b|$. Hence, we can use the fact that a double negative makes a positive to get that

$$|a + b| = -(a + b) = (-a) + (-b) < |a| + |b|.$$

Case 3. Suppose that a and b are both negative. Then $a + b$ is negative, so using the fact that a

double negative makes a positive gives us

$$|a+b| = -(a+b) = (-a)+(-b) = |a|+|b|.$$

In each case, we get that $|a+b| \leq |a|+|b|$. □

A simple induction argument using the Triangle Inequality gives us the Generalized Triangle Inequality:

Lemma A.1.2 (The Generalized Triangle Inequality)**.** *For any real numbers $a_1, a_2, \ldots, a_n$,*

$$|a_1 + a_2 + \cdots + a_n| \leq |a_1| + |a_2| + \cdots + |a_n|.$$

With the Generalized Triangle Inequality in hand, we can now prove the following theorem.

Theorem A.1.3. *Given a polynomial $p(x)$ with degree n, we get that $p(x) = O(x^n)$.*

Proof. Since p is a polynomial, we can write it as a sum of powers of x multiplied by some constant. Since p has degree n, the largest power of x is n. Hence, we may write

$$p(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$$

for some constants a_i (possibly zero) with $a_n \neq 0$ (if $a_n = 0$, then p would have degree $n-1$ or lower). Note that if $r \geq s$ and $x \geq 1$, then $x^r \geq x^s$. Using this fact, we get

$$\begin{aligned}
|p(x)| &= |a_n x^n + a_{n-1}x^{n-1} + \cdots + a_1 x + a_0| && \text{(Definition of Polynomial)} \\
&\leq |a_n x^n| + |a_{n-1}x^{n-1}| + \cdots + |a_1 x| + |a_0| && \text{(Generalized Triangle Inequality)} \\
&= |a_n| \cdot |x^n| + |a_{n-1}| \cdot |x^{n-1}| + \cdots + |a_1| \cdot |x| + |a_0| && \text{(Properties of Absolute Value)} \\
&\leq |a_n| \cdot |x^n| + |a_{n-1}| \cdot |x^n| + \cdots + |a_1| \cdot |x^n| + |a_0| \cdot |x^n| && \text{(Whenever } x \geq 1\text{)} \\
&= (|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|) \cdot |x^n|. && \text{(Distribution Law)}
\end{aligned}$$

The above computation shows us that, as long as $x \geq 1$, we get that

$$|p(x)| \leq (|a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|)|x^n|.$$

Hence, we can set $k = 1$ and $C = |a_n| + |a_{n-1}| + \cdots + |a_1| + |a_0|$ to conclude by Definition 7.1.1 that $p(x) = O(x^n)$. □

Acknowledgments

Credit original authors, institutions, and repositories (OpenStax, LibreTexts, etc.).

Attribution Index

(Optional) A chapter-by-chapter breakdown of reused materials.